Mould Tracker

System documentation

Version: web/API 2.0.0 (package.json); native shells 2.2.0
Last verified: 2026-10-29
Audience: operators, supervisors, maintenance, administrators, management, IT, developers, auditors

Generated from docs/.md. Edit the Markdown files and rebuild; do not edit this file by hand.*

Contents

If the contents are empty, right-click here and choose Update Field (or press F9).

Start 4
Mould Tracker documentation 4
Mould Tracker — complete guide 5
Introduction 8
Product overview 10
Glossary 12
Getting started 14
Prerequisites 14
First login 15
5-minute quick start 16
Navigation tour 17
User guide 19
User guide 19
Account details and passkeys 20
Install, language, theme and offline 21
Audit log 22
Extrusion dies 23
Find a mould 24
Machines 25
Manage moulds and items 26
Move a mould 28
Rack management 30
Rack view 31
Record shots 32
Report a problem 33
User access 34
Users 35
Work log report and export 36
Work log 37
Workshop queue and status changes 39
Roles 40
Roles overview 40
Permissions matrix 41
Super Admin (`super_admin`) 43
Admin (`admin`) 44
Production Supervisor (`production_supervisor`) 45

Mould Technician (`mould_technician`) — default role for new users 46
Mould Maker (`mould_maker`) 47
System Administrator (`system_admin`) 48
Workflows 49
Workflows overview 49
Mould registration and retirement 51
Mount and unmount 52
Shot count and service 53
Problem to repair 54
Workshop repair cycle 55
Out of Service 56
Data import and migration 57
Demonstrations 58
5-minute tour 58
15-minute operational demo 59
30-minute administrator / technical demo 60
Demo data and reset 61
Administration 62
Configuration 62
Operations and recovery 64
Security and access 65
Technical 67
Architecture 67
Repository structure 69
Local development 70
Database reference 71
API reference 73
Integrations 77
Deployment 78
Support 79
Troubleshooting 79
FAQ 80
Known limitations 81
Evidence 82
Evidence matrix 82
Gaps and open questions 86
Documentation changelog 87

Start

Mould Tracker documentation

Entry point and reading paths for the Mould Tracker documentation suite

Audience: Everyone · Status: Verified (static code review + automated tests) · Source: docs/README.md

Mould Tracker is a web app (installable as a PWA, with Windows and Android shells) that records **where every injection mould and extrusion die is, what condition it is in, and what work was done on it**.

> Start here: **MOULD-TRACKER-COMPLETE-GUIDE.md** — the standalone manual.

Reading paths by role

You are…	Read in this order
Production Supervisor	First login → Quick start → Find a mould → Report a problem → Mount and unmount
Mould Technician	Quick start → Move a mould → Work log → Shot count and service
Mould Maker	Quick start → Workshop queue → Workshop repair cycle → Rack view
Admin	Roles overview → Manage moulds → Machines → Users → Work log report
Super Admin	Admin path + Rack management → Out of Service → Audit log
System Administrator / IT	User access → Configuration → Security → Operations → Deployment
Developer	Architecture → Repository → Local development → API → Database

Sections

- Orientation: Introduction · Product overview · Glossary
- Getting started: Prerequisites · First login · Quick start · Navigation tour
- User guide · Roles · Workflows
- Demonstrations: 5-minute tour · 15-minute demo · Admin/technical demo · Demo data
- Administration: Configuration · Security and access · Operations and recovery
- Technical: Architecture · Repository · Local development · Database · API · Integrations · Deployment
- Support: Troubleshooting · FAQ · Known limitations
- Evidence: EVIDENCE-MATRIX · GAPS-AND-QUESTIONS · Changelog

How these docs were produced

Written from the repository source on branch `feature/mould-tracker-upgrade` on 2026-10-29. Behaviour was verified by reading the code and running the automated test suites (backend 157/157 pass, frontend 29/29 pass). The app was **not** run against a live database for this pass, so UI walkthroughs are derived from source code. No screenshots exist yet; each page lists the captures it needs.

Mould Tracker — complete guide

Standalone manual covering every verified feature, role, workflow and operation

Audience: Operators, supervisors, maintenance, administrators, management, IT, developers, auditors · Status: Verified (static code review + 186 passing automated tests); UI flows not exercised at runtime · Source: docs/MOULD-TRACKER-COMPLETE-GUIDE.md

Contents

1. What Mould Tracker is
2. Reading paths by role
3. Key concepts
4. Getting started
5. Everyday tasks
6. The rules the app enforces
7. Roles and permissions
8. Workflows
9. Administration
10. Technical reference
11. Support
12. Evidence and open questions

---

1. What Mould Tracker is

A web app (also a PWA and Windows/Android shells) that tracks every **injection mould** and **extrusion die**: where it is, whether it can be used, what it makes, how many shots it has done, and who did what to it. Details: introduction, product overview.

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart LR
    U[Browser / PWA / shells] --> N[Nginx] --> F[Next.js]
    N -->|/api| B[Express API] --> M[(MongoDB)]
```

2. Reading paths by role

Role	Read
Production Supervisor	§4, §5.1, §5.4, §5.5, role page
Mould Technician	§4, §5.1–5.6, §6, role page
Mould Maker	§4, §5.2, §5.3, §5.6, role page
Admin	§5.7, §7, §9, role page
Super Admin	Admin path + racks, audit, Out of Service (role page)
IT / System Administrator	§9, §10, role page

3. Key concepts

- **Locations:** Machine · Production Floor · Workshop · Mould Room (racks, moulds) · Tool Room (dies) · Out of Service.
- **Statuses:** Good · Needs service · Needs repair · In service · In repair.
- **Rack location IDs:** zone + row + number, for example `FLK-A3`.
- **Shots:** counted while mounted; reaching the limit makes the mould service-due.

- **History, Work log, Audit log:** append-only records.

Full list: glossary.

4. Getting started

1. Get an account from an Admin (prerequisites).
2. Sign in with email and password; set EN / ꩜ (first login).
3. Change your password in **Account details**; optionally add a passkey (HTTPS only).
4. Learn the sidebar and the mould sheet (navigation tour).
5. Do the 5-minute quick start.

5. Everyday tasks

#	Task	Who	Guide
5.1	Find a mould (Tool ID, Item ID, description, machine, rack)	All	find-a-mould
5.2	Move a mould	By permission	move-a-mould
5.3	Workshop queue, change status	Workshop roles	workshop-queue
5.4	Record shots	Supervisor, Technician, Maker*, Admins	record-shots
5.5	Report a problem	All	report-a-problem
5.6	Log work, corrections, photos	Technician, Maker, Admins	work-log
5.7	Manage moulds and items	Admins	manage-moulds
5.8	Rack view / rack management	All / Super Admin	rack-view, rack-management
5.9	Machines, users, custom access	Admins / System Administrator	machines, users, user-access
5.10	Work log report and export	Admins	work-log-report
5.11	Audit log	Super Admin	audit-log
5.12	Extrusion dies, install, theme, offline	All	extrusion-dies, app-settings-and-install

*Mould Makers record shots through `workshop.work`, but shots can only be recorded on a machine.

6. The rules the app enforces

1. Only **Good** moulds may be mounted, put on the floor or stored; anything else may only go to the Workshop.
2. **In service / In repair** only at the Workshop (or Out of Service).
3. Injection moulds are stored in the Mould Room on a rack; dies are stored in the Tool Room.
4. One mould per machine; the machine must be active.
5. Mount only on listed machines, unless an Admin overrides with a reason.
6. Racks must be active and not full; another zone's rack needs a reason.
7. Remarks are required for Workshop and Out of Service moves and for status changes.
8. Reaching the shot limit flags **Needs service** and blocks mounting; In service → Good resets the count.
9. Status changes only at the Workshop.
10. Every change writes History and the Audit log; work-log entries are never edited.

7. Roles and permissions

Role	Highlights
Super Admin	All operations, racks, audit, Out of Service
Admin	Moulds, machines, users, reports, overrides; racks read-only
Production Supervisor	Machine/floor moves, shots, problem reports
Mould Technician	Machine and workshop work, work log
Mould Maker	Storage moves, workshop work, work log
System Administrator	Everything + User access; protected account

Full matrix including who may make each move: permissions-matrix.

8. Workflows

Overview and state machine · Lifecycle · Mount/unmount · Shots and service · Problem to repair · Workshop cycle · Out of Service · Data import

9. Administration

Configuration · Security and access · Operations and recovery · Deployment

Most important operational facts:

- Production runs on Coolify. Back up MongoDB **and** the uploads folder before any migration.
- The database is shared: never delete existing data.

10. Technical reference

Architecture · Repository · Local development · Database · API (55 endpoints) · Integrations

11. Support

Troubleshooting · FAQ · Known limitations · Demos: 5 min, 15 min, 30 min, demo data

12. Evidence and open questions

EVIDENCE-MATRIX (52 claims) · GAPS-AND-QUESTIONS (9 questions) · Changelog

Introduction

What Mould Tracker is, the problem it solves and its scope

Audience: Everyone · Status: Verified (static code review + automated tests) · Source: docs/introduction.md

What is Mould Tracker?

Mould Tracker tracks the injection moulds and extrusion dies ("tools") of a plastics factory. Each tool has a **Tool ID**, a **location**, a **condition (status)**, the **items (products)** it makes, a **shot counter** and a full **history**.

Problem it solves

Without a system, moulds get lost between machines, racks and the workshop. Nobody knows which mould is due for service, and repair work goes unrecorded. Mould Tracker answers these questions at any moment:

- *Where is mould X?* On a machine, on the production floor, at the workshop, on a numbered rack, in the tool room, or out of service.
- *Can it be used?* It is Good, or it needs service/repair, or it is in service/repair.
- *Who moved it, who worked on it, and when?*

Objectives (as implemented)

Objective	How the app does it	Evidence
One trusted location per tool	Location, machine and rack rules enforced by the database model	`backend/lib/mouldState.js` `mouldStateError`
Prevent unsafe use	Only **Good** moulds leave the Workshop; one mould per machine; a reached shot limit blocks mounting	`backend/lib/workflow.js` `planMove`
Accountability	Append-only movement history, work log and audit log	`backend/lib/immutable.js`, `models/Movement.js`, `models/AuditLog.js`
Role-based control	6 roles and 19 permissions, enforced on every API call	`backend/lib/permissions.js`
Shop-floor usability	Mobile-first PWA, English/Sinhala UI, 10-second auto refresh	`frontend/src/utils/i18n.ts`, `components/MouldSearch.tsx`

Scope

In scope (verified): tool register and items; locations and moves; Mould Room zones and racks; machines; workshop status; shot counting; work log and problem reports; work-log report with CSV/XLSX export; users, roles and custom access; audit log; passkey sign-in; PWA and native shells.

Not found in the code: QR/barcode scanning, notifications (email/SMS/push), scheduled jobs, production planning, spare-parts inventory, approval chains and a general dashboard page. See known limitations.

Lifecycle overview

Diagram (Mermaid source; shown as a picture in the website version)

```
stateDiagram-v2
    [*] --> Stored: Add mould (Good)
    Stored --> InUse: Mount / to floor
    InUse --> Stored: Put back on rack
    InUse --> Workshop: Send to Workshop
    Stored --> Workshop: Send to Workshop
```

```
    Workshop --> Workshop: In service / In repair
    Workshop --> Stored: Marked Good, put on rack
    Workshop --> InUse: Marked Good, mounted
    Workshop --> OutOfService: Sent out (Super Admin)
    OutOfService --> Workshop: Returned (Super Admin)
    Stored --> Deleted: Delete (soft)
    Deleted --> Stored: Restore
```

"Stored" means Mould Room (injection moulds) or Tool Room (extrusion dies). "In use" means on a machine or on the Production Floor.

Next: Product overview · Glossary

Product overview

Users, platforms, architecture, requirements and the end-to-end process

Audience: Everyone; managers; IT · Status: Verified (static code review) · Source: docs/product-overview.md

Who uses it

Six roles (see Roles): Super Admin, Admin, Production Supervisor, Mould Technician, Mould Maker, and the protected System Administrator.

Platforms

Platform	Status	Evidence
Web browser (desktop and phone), responsive layout	Verified	`frontend/` (Next.js app)
Installable PWA with an offline page	Verified	`frontend/src/app/manifest.ts`, `public/sw.js`, `public/offline.html`
Windows desktop shell (Electron, unsigned installer)	Verified (source)	`native/desktop/`, `native/README.md`
Android shell (WebView APK, Android 8+)	Verified (source)	`native/android/`
Apple shell	Source only, no release	`native/apple/`, `native/README.md`

Native shells only open the server URL; they hold no data. Passkeys are not supported inside the native WebViews (`native/README.md`).

Architecture

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart LR
    U[Browser / PWA / native shell] -->|HTTPS| N[Nginx proxy]
    N -->|pages| F[Next.js frontend]
    N -->|/api| B[Express API]
    B --> M[(MongoDB replica set)]
    B --> D[(Uploads folder - WebP pictures)]
```

- **Frontend:** Next.js 16, React 19, Tailwind CSS 4, TypeScript.
- **Backend:** Node.js, Express 5, Mongoose 8, Zod validation, JWT session in an HttpOnly cookie, WebAuthn passkeys, Sharp image processing, ExcelJS export.
- **Database:** MongoDB with multi-document transactions, so a replica set is required. Docker Compose stack in `compose.yaml`.

Details: technical/architecture.md.

End-to-end process

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart TD
    A[Admin adds mould and items] --> B[Mould on a rack in the Mould Room]
    B -->|Technician mounts| C[On a machine]
    C -->|Record shots| C
    C -->|shots reach limit| D[Flagged Needs service]
    C -->|Anyone reports a problem| C
    D -->|Send to Workshop| E[Workshop]
    E -->|In service / In repair, work logged| E
    E -->|Mark Good| F{Where next?}
    F --> B
    F --> C
    G[Admin: work log report and export] -.reads.-> E
```

System requirements

- **Server:** Docker with Compose, or Node.js plus a MongoDB replica set. See Deployment.
- **Client:** a current browser. Passkeys need a trusted HTTPS address.

Known limitations

See support/known-limitations.md.

Glossary

Terms used in Mould Tracker and in these docs

Audience: Everyone · Status: Verified (static code review) · Source: docs/glossary.md

Term	Meaning	Code value
Mould / tool	One Tool ID: an injection mould or an extrusion die.	`Mould` model
Tool ID	Unique identifier, stored uppercase (for example `FLK19066`). Searches ignore spacing.	`toolId`
Tool type	Injection mould (default) or extrusion die.	`INJECTION_MOULD`, `EXTRUSION_DIE`
Item	A product a mould makes: Item ID, description, cavities, material, masterbatch, QAS, pictures.	`Item` model
Location	Machine, Production Floor, Workshop, Mould Room, Tool Room or Out of Service.	`MACHINE`, `PRODUCTION_FLOOR`, `WORKSHOP`, `MOULD_ROOM`, `TOOL_ROOM`, `OUT_OF_SERVICE`
Mould Room	Storage for injection moulds, organised into zones and racks.	`MOULD_ROOM`
Tool Room	Storage for extrusion dies. It has no racks.	`TOOL_ROOM`
Zone	A side of the Mould Room (for example FLK). Also used as a mould's **owner**.	`Zone.code`
Rack / rack location	Zone + row + number gives a location ID such as `FLK-A3`. Capacity is optional.	`Rack.locationId`
Awaiting rack	An imported mould in the Mould Room with no rack yet ("No rack yet").	`awaitingRack`
Status	Good, Needs service, Needs repair, In service, In repair.	`GOOD`, `NEEDS_SERVICE`, `NEEDS_REPAIR`, `IN_SERVICE`, `IN_REPAIR`
Capable machines	Machines a mould is listed for. Other machines need an Admin override with a reason.	`capableMachines`
Shots / shot limit	Cycles counted while mounted. Reaching `shotsToService` makes the mould service-due.	`totalShots`, `shotsToService`
Workshop queue	Moulds at the Workshop plus moulds flagged elsewhere.	`/workshop`
Work log	Append-only record of work: repair, service, minor fix, inspection, cleaning, other.	`WorkLog`
Problem report	A work-log entry anyone may make. It flags the mould but never changes status or location.	`problemReported`, `openProblem`
Correction	A new work-log entry pointing at a wrong one. Entries are never edited.	`correctsId`
Out of Service	Sent outside the factory for repair with recipient, address and repair details. Super Admin only.	`OUT_OF_SERVICE`
Movement	One history line: created, edited, moved, status changed, deleted, restored, imported.	`Movement`
Audit log	Append-only who / what / before / after record of every change.	`AuditLog`
Override	An Admin decision to mount on a non-listed machine, or to store on another zone's rack. A reason is required.	`overrideReason`
Passkey	Fingerprint, face or PIN sign-in using WebAuthn.	`Passkey`
Private user	How a protected System Administrator appears to everyone else.	`backend/lib/accountPrivacy.js`

Term	Meaning	Code value
Version (optimistic lock)	A number sent with edits so the server can detect that someone else changed the mould first.	`version`

Getting started

Prerequisites

What a user needs before signing in to Mould Tracker

Audience: All users; support · Status: Verified (static code review) · Source: docs/getting-started/prerequisites.md

Goal: be ready to sign in on your first day.

Before you begin

You need	Why	Evidence
An account (email + password) created by an Admin, Super Admin or System Administrator	There is no self-registration	`backend/routes/userRoutes.js` (`users.manage`)
A role chosen for you	Your role decides what you can do	`backend/lib/permissions.js`
The factory app address (ask your supervisor/IT)	The app runs on the factory server	`deploy/README.md`
Network access to that server (factory LAN)	Native shells and the PWA need the server	`native/README.md`
A current browser, or the Windows / Android app	Supported clients	Product overview
Optional: trusted HTTPS and a device with fingerprint/Face ID/PIN	Needed only for passkey sign-in	`backend/utils/passkeyConfig.js`
Optional: the factory certificate installed on the device	The LAN uses a local certificate authority	`deploy/LOCAL-HTTPS.md`

Passwords

- Minimum 8 characters, maximum 128 (`backend/validators/schemas.js` `password`).
- Too many sign-in attempts are throttled: 40 per 10 minutes per client (`backend/routes/authRoutes.js` `loginLimit`).

Expected result

You have an email, a temporary password and the app address.

Related: First login · Security and access

First login

Sign in, set your language and theme, change your password and optionally add a passkey

Audience: All users · Status: Verified (static code review); not walked through on a running instance · Source: docs/getting-started/first-login.md

Goal: sign in, secure your account and land on **Find a mould**.

Before you begin

See Prerequisites.

Steps

1. Open the app address. Signed-out users see the sign-in card ("Sign in to see where every mould is.").
2. *(Optional)* Choose **EN** or ಐ at the top of the card, and the light/dark theme toggle.
3. Enter **Email** and **Password**, then select **Sign in**.
4. You arrive on **Find a mould** (`/find`), the default page.
5. Open **Account details** from the sidebar. Enter your **Current password**, a **New password (min 8 characters)**, repeat it, and select **Save changes**.
6. *(Optional, HTTPS only)* Open **Passkeys / Biometric login** in the sidebar. Enter a device name and your current password, then select **Add a passkey**. Next time you can use **Sign in with fingerprint / Face ID**.

> Screenshot needed: sign-in card in light theme, English, with the passkey button visible (`/login`). Screenshot needed: Account details page (`/account`) with the password fields.

Expected result

- You are signed in. The session is an HttpOnly cookie valid for 7 days (`backend/utils/session.js`).
- Your language choice is saved to your account and follows you to other devices (`PATCH /api/auth/me/language`).

Troubleshooting

Symptom	Cause	Fix
"Invalid email or password"	Wrong credentials, or the account is disabled	Retry; ask an Admin to check the account is active
"Too many authentication attempts…"	Rate limit (40 per 10 min)	Wait 10 minutes
Passkey button disabled; note says passkeys need trusted HTTPS	Server not configured for passkeys, or plain HTTP	Use password; IT sets `WEBAUTHN_ORIGIN` / `WEBAUTHN_RP_ID`
"Passkey request was cancelled or timed out."	Prompt dismissed	Try again
Signed out unexpectedly	Account disabled/deleted, or session expired	Sign in again; contact an Admin

Related: Navigation tour · Quick start · Account and passkeys

5-minute quick start

Find a mould, read its state, and take your first action

Audience: New users (any role) · Status: Verified (static code review); untested on a running instance · Source: docs/getting-started/quick-start.md

Goal: find a mould, understand where it is and what state it is in, and do one thing your role allows.

Before you begin

You are signed in (First login).

Steps

1. **Find it.** On **Find a mould**, type part of a Tool ID, an Item ID, a product description, a machine code or a rack (for example `FLK A3`). *Checkpoint:* the result count updates ("1 mould" / "N moulds").
2. **Read the card.** The status badge shows the condition (Good / Needs service / …). The location line shows the machine, rack, Production Floor, Workshop, Tool Room or Out of Service.
3. **Open it.** Tap the card. Check **Items made by this mould**, **Shots**, and **History**.
4. **Take your first action**, depending on your role:

Role	Try this	Doc
Production Supervisor	**Report a problem** with a short description	Report a problem
Mould Technician	**Mount on a machine** (choose a machine listed for the mould)	Move a mould
Mould Maker	Open **Workshop queue** and **Change status** on a mould at the Workshop	Workshop queue
Admin	**Add mould** with one item	Manage moulds

5. **Confirm.** A success message appears, the card updates, and a new line appears in **History**.

Expected result

You can locate any mould in under 30 seconds and see the actions available to you.

Troubleshooting

- *"…can only go to the Workshop"*: the mould is not Good. Only Good moulds may be mounted or stored.
- *"…already has …; take it off first"*: the machine is occupied. Move that mould off first.
- More in Troubleshooting.

First-day checklist

- ☐ Signed in and changed password
- ☐ Language set (EN / සිං)
- ☐ Found three moulds by Tool ID, Item ID and rack
- ☐ Opened a mould's history
- ☐ Performed one action allowed for your role
- ☐ *(Optional)* Installed the app (**Install app**) or added a passkey

Navigation tour

The sidebar, pages, search, mould cards and the mould detail sheet

Audience: All users · Status: Verified (static code review) · Source: docs/getting-started/navigation-tour.md

Sidebar

The sidebar collapses on desktop (**Expand menu / Collapse menu**) and becomes a menu on phones. Links only appear when you hold the permission (`frontend/src/components/Sidebar.tsx`, `frontend/src/utils/permissions.ts` `canVisitPage`).

Link	Route	Shown when you have	Purpose
Find a mould	`/find`	`moulds.view`	Search and browse injection moulds
Extrusion die	`/extrusion`	`moulds.view`	The same screen for extrusion dies
Rack view	`/racks`	`racks.view`	Mould Room map by zone and row
Workshop queue	`/workshop`	`moulds.view`	Moulds at the Workshop or flagged elsewhere
User access	`/access`	`access.manage` (System Administrator)	Custom permissions per user
Machines	`/machines`	`moulds.manage`	Machine list, add, rename, enable/disable
Users	`/users`	`users.manage`	Create, edit, disable, delete users
Work log report	`/work-report`	`worklog.report`	Search, summarise and export work
Rack management	`/rack-admin`	`racks.manage`, or `moulds.manage` (view only)	Zones and racks
Audit log	`/audit`	`audit.view`	Every change with before/after
Account details	`/account`	everyone	Name, email, password
Passkeys / Biometric login	`/passkeys`	everyone	Manage passkeys
Sign out	—	everyone	End the session

Opening a page without permission shows nothing useful, and the API refuses the data with HTTP 403.

Find a mould

- **Search box:** "Tool ID, Item ID, description, machine or rack". Spacing is ignored (`FLK 19066` finds `FLK19066`); a rack like `FLK A3` finds rack `FLK-A3`.
- **Filter chips** show counts (All, Service / repair, No rack yet, by location and status) from `GET /api/moulds/stats`.
- **Cards** show Tool ID, status badge, location line, item count, picture thumbnail, last work and "Service due".
- The list refreshes **every 10 seconds**. The sync indicator shows "Live — synced with server" or "Offline — cannot reach server".

Mould detail sheet

Tap a card to open the sheet (`?mould=<id>` in the URL, so it can be bookmarked). It shows:

- Items made by this mould (pictures open in a lightbox), capable machines, shots, last serviced, owner (zone), tool set, size, notes, latest remarks.
- **Move or update** actions you are allowed to take (others are hidden; if none: "You can view this mould but not move it.").

- Work log, **Report a problem**, and **History**.
- For Admins: **Edit mould**, **Delete mould**, **Restore mould**.

> Screenshot needed: Find page on a phone with filter chips and two cards. Screenshot needed: Mould detail sheet showing actions and history.

Related: User guide

User guide

User guide

Index of verified features, one page each

Audience: All users · Status: Verified (static code review) · Source: docs/user-guide/README.md

Each page follows the same layout: Availability, Role permissions, Navigation, Fields, Actions, Rules, State changes, Errors, Examples and Evidence.

Feature	Page	Who
Search and browse moulds / dies	find-a-mould.md	Everyone
Extrusion dies	extrusion-dies.md	Everyone
Add, edit, delete, restore moulds and items	manage-moulds.md	Admin, Super Admin
Move a mould (machine, floor, rack, workshop)	move-a-mould.md	By permission
Workshop queue and status change	workshop-queue.md	Workshop roles
Record shots	record-shots.md	Machine and workshop roles
Work log (log work, corrections, photos)	work-log.md	Technician, Maker, Admins
Report a problem	report-a-problem.md	Everyone
Rack view	rack-view.md	Everyone
Rack management (zones, racks)	rack-management.md	Super Admin (Admin read-only)
Machines	machines.md	Admin, Super Admin
Users	users.md	Admin, Super Admin
User access (custom permissions)	user-access.md	System Administrator
Work log report and export	work-log-report.md	Admin, Super Admin
Audit log	audit-log.md	Super Admin
Account details and passkeys	account-and-passkeys.md	Everyone
Install app, language, theme, offline	app-settings-and-install.md	Everyone

Account details and passkeys

Change your name, email and password; add and remove passkeys

Audience: All users · Status: Verified (static code review + tests) · Source: docs/user-guide/account-and-passkeys.md

Account details (`/account`)

- Change **Name**, **Email** and optionally **New password (min 8 characters)** (repeat it).
- **Current password** is always required. Leave the new password blank to keep the current one.
- Errors: "Current password is incorrect", "The two new passwords do not match", "Email already in use".

Passkeys (`/passkeys`)

- **Add a passkey:** give a **Device name** and your **Current password**, then follow the device prompt (fingerprint, Face ID or PIN).
- **Remove:** requires your current password.
- Only the public key is stored on the server; biometric data stays on the device.
- Requires the server to be configured with a trusted HTTPS origin. Otherwise the page shows "Passkeys need a trusted HTTPS address and a supported browser." Password sign-in always remains available.
- Not available in the Android/Windows shells (`native/README.md`).

Evidence

`backend/controllers/authController.js` `updateProfile`; `backend/controllers/passkeyController.js`; `backend/utils/passkeyConfig.js`; `frontend/src/app/(tracker)/account/page.tsx`, `passkeys/page.tsx`; `backend/test/profile.test.js`, `auth.test.js`.

Install, language, theme and offline

Install the PWA or native app, switch language and theme, behaviour when offline

Audience: All users · Status: Verified (static code review + tests) · Source: docs/user-guide/app-settings-and-install.md

Language

EN / සිං toggle (sign-in card and sidebar). It is saved to your account, so it follows you to other devices. Sinhala uses the Noto Sans Sinhala font.

Theme

Light / dark toggle. The choice is stored on the device (`mould-tracker-theme`); otherwise the system preference is used.

Install

- **Install app** button (PWA). On iPhone/iPad: "In Safari, choose Share → Add to Home Screen…"
- Windows installer and Android APK: built by IT from `native/` (see Deployment).

Offline

The service worker shows an offline page when the server cannot be reached, and the sync indicator shows "Offline — cannot reach server". **Changes cannot be made offline**; there is no offline queue.

Old links

Bookmarks or QR links in the old `#/page/m/<id>` format are redirected to the new pages (`getLegacyDestination`).

Evidence

`frontend/src/utils/i18n.ts`; `frontend/src/utils/helpers.ts`; `frontend/public/sw.js`, `offline.html`; `frontend/src/providers/PwaProvider.tsx`; `frontend/test/service-worker.test.mjs`, `theme.test.mjs`.

Audit log

Review every change with actor, before/after and reason

Audience: Super Admin, System Administrator, auditors · Status: Verified (static code review) · Source: docs/user-guide/audit-log.md

Availability

Route `/audit`, requires `audit.view` (Super Admin; System Administrator).

What you see

Entries newest first, filterable by **Type** (Moulds, Items, Racks, Zones, Machines, Users, Imports, Migration). Each shows actor, action, label (Tool ID / rack location / email), **Before**, **After**, **Reason**. **Load more** pages through them ("n of total"); the API also supports action, actor, entity and date filters, with up to 200 per page.

Recorded actions (verified)

`mould.created/updated/moved/status_changed/shots_recorded/deleted/restored`, `item.created/updated/deleted`, `machine.created/updated`, `zone.created/updated/deleted`, `rack.created/updated/renamed/deleted/bulk_generated/row_removed`, `user.created/updated/deleted`, `user.access.updated`, `worklog.created/corrected/exported`, `problem.resolved`, plus import and migration runs.

Guarantees

Audit, movement and work-log records are **append-only**: the model plugin blocks updates and deletes through the application. Entries involving a protected System Administrator are hidden from other users.

Evidence

`backend/models/AuditLog.js`; `backend/lib/immutable.js`; `backend/lib/audit.js`; `backend/controllers/auditController.js`; `frontend/src/app/(tracker)/audit/page.tsx`.

Extrusion dies

How extrusion dies differ from injection moulds

Audience: All users · Status: Verified (static code review + tests) · Source: docs/user-guide/extrusion-dies.md

Availability

Route `/extrusion`, requires `moulds.view`. The screen is the same search screen as Find a mould, filtered to `toolType = EXTRUSION_DIE`.

Rules that differ

Rule	Injection mould	Extrusion die
Storage location	Mould Room (on a rack)	**Tool Room** (no rack)
Moving to the wrong store	Refused: "Use the Tool Room for extrusion dies and the Mould Room for injection moulds"	Same
Workshop, machine, floor, shots, work log	Same rules	Same rules

Moving into or out of the Tool Room needs `moves.mouldRoom`, the same as the Mould Room.

Evidence

`backend/lib/mouldState.js` (`storageLocation`, `isStorage`); `backend/migrations/moveExtrusionToToolRoom.mjs`; `backend/test/toolRoom.test.js`; `frontend/src/app/(tracker)/extrusion/page.tsx`.

Find a mould

Search, filter and open injection moulds

Audience: All users · Status: Verified (static code review) · Source: docs/user-guide/find-a-mould.md

Availability

Route `/find` (also the default page). Requires `moulds.view`, which every role has by default.

Navigation

Sidebar → **Find a mould**.

Fields and controls

Control	Behaviour
Search ("Tool ID, Item ID, description, machine or rack")	Case-insensitive. Matches Tool ID (ignoring spaces), mould notes, Item ID, item description, machine code or name, rack location (for example `FLK A3` → `FLK-A3`) and capable machines.
Filter chips	Counts by location, **Service / repair** (Needs/In service/repair), **No rack yet**. Select a chip to filter; **Show all** clears it.
Cards	Tool ID, status badge, location, number of items, first picture, last work and **Service due** marker.
Sorting	Tool ID in natural numeric order (`FLK2` before `FLK10`).
Refresh	Automatically every 10 s; sync indicator shows Live / Offline.

There is no pagination: all matching moulds are returned.

Actions

Select a card to open the mould detail sheet. Admins also see **Add mould** (Manage moulds).

Rules

- Deleted moulds are hidden. Admins can list them with the API filter `deleted=true`.
- `/find` shows injection moulds only; extrusion dies are on `/extrusion`.

Examples

- `19066` → finds `FLK19066`.
- `INJ-03` → moulds on, or listed for, machine INJ-03.

Errors

"No mould matches your search." means no results. "Offline — cannot reach server" means a network problem; see troubleshooting.

> Screenshot needed: `/find` with search text `FLK` and the Service / repair chip selected.

Evidence

`backend/controllers/mouldController.js` `listMoulds`, `mouldStats`; `frontend/src/components/MouldSearch.tsx`; `frontend/src/utils/i18n.ts` (`search`, `dash`).

Machines

View machines and their mounted moulds; rename and enable/disable machines

Audience: Admin, Super Admin · Status: Verified (static code review) · Source: docs/user-guide/machines.md

Availability

Page `/machines` requires `moulds.manage`. The API list (`GET /api/machines`) only needs `moulds.view`, because the move sheet uses it.

Actions in the UI

- Search by machine code, name or mounted mould.
- **View / edit details:** change **Machine name** and **Active / Inactive**.
- The machine code is fixed: "Machine code is fixed to preserve mould compatibility records."

Rules

- A machine with a mounted mould cannot be disabled (`MACHINE_OCCUPIED`): "Take the mould off this machine before disabling it."
- Inactive machines cannot receive moulds.

Not in the UI

Creating a machine is supported by the API (`POST /api/machines`, `moulds.manage`), but the `/machines` page has no add button. Machines are otherwise created by the seed and import scripts. See GAPS.

Evidence

`frontend/src/app/(tracker)/machines/page.tsx`; `backend/controllers/machineController.js`; `backend/routes/machineRoutes.js`.

Manage moulds and items

Add, edit, delete and restore moulds; manage items and item pictures

Audience: Admin, Super Admin · Status: Verified (static code review + tests) · Source: docs/user-guide/manage-moulds.md

Availability and role permissions

Requires `moulds.manage`, held by Super Admin, Admin and System Administrator. Other roles do not see these buttons, and the API returns 403.

Navigation

- **Add mould:** on Find a mould / Extrusion die.
- **Edit mould / Delete mould / Restore mould:** in the mould detail sheet.

Fields — mould

Field	Required	Rules
Tool ID	Yes	Max 100, stored uppercase, unique. A duplicate that differs only in spacing is refused (`DUPLICATE_TOOL_ID`).
Tool type	No	Injection mould (default) or extrusion die
Tool set count, Size	No	Max 50 characters each
Owner (zone)	No	Zone code, letters and digits, max 10
Capable machines	No	Comma separated, max 50; codes are normalised
Shots to service	No	Whole number ≥ 0; empty means no limit
Notes	No	Max 2000
Starts at	Create only	Mould Room (default), Tool Room, Production Floor or Workshop. Never a machine. Mould Room needs a rack.
Items	No	Up to 100 at creation

Fields — item

Item ID (required, uppercase, unique within the mould), Description (500), Cavities (whole number), QAS, Material ID and description, Masterbatch ID and description, Notes (1000), Sort order, and a **picture** (edit only; JPEG/PNG/WebP, saved as an 800 px WebP plus a 200 px thumbnail).

Actions

1. **Add:** fill the form, choose **Starts at** (and a rack for the Mould Room), add items, then save. The status is always **Good**.
2. **Edit:** change fields and save. The form sends the mould's `version`; if someone else changed it first you see "…was changed by someone else. Refresh and try again."
3. **Delete:** enter an optional reason and confirm. This is a soft delete: history and items are kept.
4. **Restore:** open a deleted mould and select **Restore mould**.

Rules and state changes

- A mould **on a machine cannot be deleted**: "Take … off the machine before deleting it".
- Restore fails if another mould has since taken the Tool ID.
- Changing Shots to service can immediately flag the mould **Needs service** (rule 8) when it is outside storage.

- Every change writes a History line (`CREATED`, `EDITED`, `DELETED`, `RESTORED`) and an Audit log entry.

Errors

Code	Meaning
`DUPLICATE_TOOL_ID`	Tool ID exists (or exists deleted: restore it instead)
`STALE`	Someone else saved first
`NO_CHANGE`	Nothing changed
`UNMOUNT_FIRST`	Mould is on a machine
`INVALID_PHOTO` / `TOO_LARGE`	Picture unreadable or too big

> Screenshot needed: Add mould form with one item filled in.

Evidence

`backend/controllers/mouldController.js` (`createMould`, `updateMould`, `deleteMould`, `restoreMould`, `addItem`, `updateItem`, `deleteItem`); `backend/validators/schemas.js`; `backend/lib/images.js`; `frontend/src/components/MouldFormSheet.tsx`; `backend/test/itemPhoto.test.js`, `mouldModel.test.js`.

Move a mould

Mount on a machine, move to the floor, put on a rack, send to the Workshop

Audience: Production Supervisor, Mould Technician, Mould Maker, Admins · Status: Verified (static code review + tests) · Source: docs/user-guide/move-a-mould.md

Availability

Mould detail sheet → **Move or update**. Only the actions you may take are shown (`frontend/src/utils/permissions.ts` `canMoveTo`); the server checks again.

Actions

Button	Destination	Permission needed
Mount on a machine / Move to another machine	Machine	`moves.machine` (+ `moves.mouldRoom` if it is coming from storage)
To Production Floor	Production Floor	`moves.machine` (from the Workshop: `moves.machine` **or** `workshop.work`; from storage: `moves.mouldRoom`)
Put in Mould Room / Change rack	Mould Room rack	`moves.mouldRoom`
Put in Tool Room	Tool Room (dies)	`moves.mouldRoom`
Send to Workshop	Workshop	`workshop.work` (+ `moves.mouldRoom` if from storage)
Send out of service / Return to Workshop	Out of Service	`moves.outOfService` (Super Admin only) — see Out of Service

Fields

- **Machine:** listed machines first ("Listed for this mould"), then "Other machines". Each shows "free" or "has X".
- **Rack:** shows occupancy `n/capacity`.
- **New status:** only when sending to the Workshop ("Keep current status" or choose one).
- **Remarks:** required when sending to the Workshop or Out of Service.
- **Reason for override:** required for a non-listed machine (Admins only) or another zone's rack.

Rules

1. Only **Good** moulds may go anywhere except the Workshop/Out of Service (`NOT_GOOD`).
2. One mould per machine (`MACHINE_OCCUPIED`); take the other one off first.
3. Machine must be active (`MACHINE_INACTIVE`).
4. A mould at its shot limit cannot be mounted (`SERVICE_DUE`).
5. Non-listed machine: refused for non-Admins (`NOT_CAPABLE`); Admins must give a reason.
6. Rack must be active and not full (`RACK_INACTIVE`, `RACK_FULL`). A mould whose owner differs from the rack's zone needs a reason (`OWNER_MISMATCH`).
7. Same place again is refused (`NO_CHANGE`).
8. Sending to the Workshop with remarks also **closes an open problem report**.

State changes

Location, machine and rack are updated; `lastMovedAt`/`lastMovedBy` are set; a `MOVED` history line and an `mould.moved` audit entry are written in one transaction. Overrides are stored with the reason.

Example

Technician mounts `FLK19066` on `INJ-03`: Mould Room rack `FLK-A3` → Machine `INJ-03`. Needs both `moves.mouldRoom` and `moves.machine`; the Technician role has only `moves.machine`, so a **Mould Maker or Admin** must first take it to the floor, or an Admin mounts it. See roles.

> Screenshot needed: Move sheet with the machine list showing "Listed for this mould" and "has X".

Evidence

`backend/lib/workflow.js` (`movePermissionGroups`, `planMove`); `backend/controllers/mouldController.js` `moveMould`; `backend/test/workflow.test.js` ("who may move what"); `frontend/src/components/MouldActions.tsx`.

Rack management

Create and maintain Mould Room zones and racks

Audience: Super Admin (Admin view-only) · Status: Verified (static code review + tests) · Source: docs/user-guide/rack-management.md

Availability and role permissions

Route `/rack-admin`. Changes require `racks.manage` (Super Admin). Admins (`moulds.manage`) see it read-only: "View only — only a Super Admin can change zones and racks."

Zones

Action	Rules
Add zone	Code: letters and digits, max 10, unique; Name required; Order
Rename / Change code	Changing the code renames every rack location ID in that zone
Disable / Enable	A disabled zone's racks cannot receive moulds
Delete	Refused while any rack in the zone holds moulds (`RACK_OCCUPIED`)

Racks

Action	Rules
Add one rack	Zone, Row (1–3 letters), Number (1–999), Capacity (empty = unlimited), Note (300). Location ID = `ZONE-ROWNUMBER`, e.g. `FLK-A3`.
Bulk generate racks	Rows from…to, numbers from…to, capacity. **Preview** first shows "n new / n restored / n already exist", then **Create n racks**.
Edit rack	Row, number, capacity, in use, note. A code change is audited as `rack.renamed`.
Delete rack	Refused while it holds moulds: move them or disable the rack.
Delete row	Refused while any rack in the row holds moulds.

Racks are soft-deleted; creating the same code again revives it.

Evidence

`backend/controllers/rackController.js`; `backend/models/Rack.js`, `Zone.js`; `backend/lib/racks.js`; `frontend/src/app/(tracker)/rack-admin/page.tsx`.

> Screenshot needed: Bulk generate preview for zone FLK rows A–B, numbers 1–5.

Rack view

Visual map of the Mould Room by zone, row and rack

Audience: All users · Status: Verified (static code review) · Source: docs/user-guide/rack-view.md

Availability

Route `/racks`, requires `racks.view` (every role).

What you see

- Zone filter: **All zones** or one zone ("FLK side").
- Rows ("Row A") of racks, coloured: **empty rack**, **in use**, **full** (at capacity). Inactive racks show "Not in use".
- **Waiting for a rack:** moulds in the Mould Room with no rack yet (from import). Open one and choose a rack.
- Select a rack to see "Moulds on FLK-A3".

Evidence

`frontend/src/app/(tracker)/racks/page.tsx`; `frontend/src/utils/helpers.ts` `rackTone`; `backend/controllers/rackController.js` `listRacks`.

> Screenshot needed: `/racks` with one zone, mixed empty / in use / full racks.

Record shots

Add produced shots to a mounted mould and trigger service-due

Audience: Production Supervisor, Mould Technician, Mould Maker, Admins · Status: Verified (static code review + tests) · Source: docs/user-guide/record-shots.md

Availability

Mould detail sheet → **Record shots**. Requires `moves.machine` or `workshop.work`.

Fields

Shots produced: whole number from 1 to 10,000,000. The number is **added** to the running total.

Rules and state changes

- The mould must be **on a machine** (`NOT_ON_MACHINE`).
- When `totalShots ≥ shotsToService` and the mould is Good, it becomes **Needs service** automatically. A `STATUS_CHANGED` history line says "Shot limit reached (n/limit shots)".
- After that it can only go to the Workshop. A completed service resets the count. See Shot count and service.

Example

Limit 100,000, total 98,000. Record 3,000 → total 101,000 → **Needs service**; the card shows **Service due**.

Evidence

`backend/lib/workflow.js` `planShots`, `serviceDueChange`;
`backend/controllers/mouldController.js` `recordShots`.

Report a problem

Flag an issue on a mould without changing its status or location

Audience: Everyone (especially Production Supervisors) · Status: Verified (static code review + tests) · Source: docs/user-guide/report-a-problem.md

Availability

Mould detail sheet → **Report a problem**. Requires `problem.report`, which every role has.

Steps

1. Open the mould.
2. Select **Report a problem**, describe it (required, max 2000), and optionally add up to 3 photos and the time it happened.
3. Submit.

Expected result

- The mould gets a problem badge and appears under problem reports on the Workshop queue.
- **Status and location do not change.**
- A work-log entry (type Inspection, marked "problem reported") is stored with you as reporter.

How a problem is closed

Automatically, when someone **logs work** on the mould or **sends it to the Workshop**. Both write an audit entry `problem.resolved`.

Evidence

`backend/lib/workLog.js` `planProblemReport`; `backend/controllers/workLogController.js` `reportProblem`, `resolveProblem`; `backend/controllers/mouldController.js` `moveMould`; `frontend/src/components/ProblemForm.tsx`.

User access

Replace a user's role defaults with a custom permission set

Audience: System Administrator · Status: Verified (static code review + tests) · Source: docs/user-guide/user-access.md

Availability

Route `/access`, requires `access.manage`. Only the System Administrator role has it, and it can never be granted.

Steps

1. Find a user by name or email and select them.
2. The page shows **Role defaults**. Switch to **Custom access** and tick permissions (labels such as "Record work", "View and export work reports").
3. Save. **Reset to role defaults** removes the custom set.

Rules

- A custom set **replaces** the role defaults entirely. An empty set allows only sign-in and managing your own account.
- `moves.outOfService` can only be given to Super Admins.
- System Administrator accounts cannot be restricted.
- Effective immediately on the server; the UI refreshes permissions on focus and every minute.
- Audited as `user.access.updated`.

Evidence

`backend/controllers/userController.js` `listUserAccess`, `updateUserAccess`; `backend/lib/permissions.js` `can`; `frontend/src/app/(tracker)/access/page.tsx`; `frontend/src/components/AppLayout.tsx` (60 s refresh); root `README.md`.

Users

Create, edit, disable and delete user accounts

Audience: Admin, Super Admin, System Administrator · Status: Verified (static code review + tests) · Source: docs/user-guide/users.md

Availability

Route `/users`, requires `users.manage` (Super Admin, Admin, System Administrator).

Fields

Name (max 120), Email (unique), Password (8–128, create or reset), Role (default Mould Technician), Active.

Who can manage whom

Actor	Can manage	Can assign roles
Admin	Everyone except Super Admins and System Administrators	Everything except Super Admin and System Administrator
Super Admin	Everyone except System Administrators	Everything except System Administrator
System Administrator	Everyone, but other System Administrator accounts are invisible	All roles

Rules

- You cannot disable yourself, change your own role or delete yourself.
- **Delete** is soft: the account is marked deleted and inactive; its history stays.
- Changes take effect on the user's **next request**; there is no need to wait for the session to expire.
- Every action is audited (`user.created`, `user.updated`, `user.deleted`).

Evidence

`backend/controllers/userController.js`; `backend/lib/permissions.js` `canManageUser`; `frontend/src/utils/permissions.ts` `assignableRoles`; `backend/test/auth.test.js`.

Work log report and export

Search, filter, summarise and export work-log entries

Audience: Admin, Super Admin · Status: Verified (static code review + tests) · Source: docs/user-guide/work-log-report.md

Availability

Route `/work-report`, requires `worklog.report` (Super Admin, Admin).

Filters

Filter	Values
Search	Text (also matches the mould's Item IDs)
Tool ID, Person, Machine	Free text / person
Type	Repair, Service, Minor fix, Inspection, Cleaning, Other
Outcome	OK, Needs repair, Needs service, None
Status	Any mould status
Problem reports only	yes/no
Range	Today, 7 days, 30 days, This month, All, Custom (whole local days, Asia/Colombo)

Your last filters are remembered on this device.

Outputs

- **List:** paged on the server (20 per page by default, max 100). A table on wide screens, cards on phones; search words are highlighted; corrected entries are marked.
- **Summary:** per person (each named person counts once per entry) and per mould.
- **Export:** CSV or XLSX. The XLSX has a "Work log" sheet and an "About this export" sheet listing the filters. **Maximum 5,000 rows**, otherwise `EXPORT_TOO_LARGE` asks you to narrow the filters. Each export is audited (`worklog.exported`).

Protected System Administrator identities appear as **Private user** in results and exports.

Evidence

`backend/controllers/workLogController.js` (`listWorkLogs`, `workLogSummary`, `exportWorkLogs`); `backend/lib/workLogFilter.js`; `backend/lib/dateRange.js`; `frontend/src/app/(tracker)/work-report/page.tsx`; `backend/test/workLogReport.test.js`.

Work log

Record repair, service and other work on a mould, with photos and corrections

Audience: Mould Technician, Mould Maker, Admins · Status: Verified (static code review + tests) · Source: docs/user-guide/work-log.md

Availability and role permissions

- **View** a mould's work log: `worklog.view` (everyone).
- **Log work:** `worklog.create`, held by Super Admin, Admin, Mould Technician and Mould Maker. Work can be logged **wherever the mould is**, including on a machine.

Fields

Field	Required	Rules
Type	Yes	Repair, Service, Minor fix, Inspection, Cleaning, Other
What was done (description)	Yes	Max 2000
Parts used	No	Max 1000
Time spent (minutes)	No	0–100,000
Done by	No (defaults to you)	1–5 people, each allowed to do mould work
Done at	No (defaults to now)	Cannot be more than 2 minutes in the future
Shots (counter reading)	No	Whole number ≥ 0
Outcome	No	OK, Needs repair, Needs service
Send to Workshop	No	Only with a Needs… outcome
Photos	No	Up to 3; JPEG/PNG/WebP, shrunk in the browser, max 6 MB each

What the outcome does

Outcome	Effect
OK on a non-Good mould	Becomes **Good**, unless the shot limit is still reached and the type is not Service (warning `SERVICE_STILL_DUE`)
OK with type **Service**	Also resets shots to 0 and sets Last serviced
Needs repair / Needs service + Send to Workshop	Moves it to the Workshop with that status, if you may make the move (otherwise warning `WORKSHOP_MOVE_NOT_ALLOWED`)
Needs… without sending, on a machine or floor	Status flagged in place
Needs… while in storage	Entry saved, status unchanged, warning `WORKSHOP_MOVE_REQUIRED`

The work itself never moves the mould; only **Send to Workshop** does.

Corrections

Entries cannot be edited or deleted (append-only). To fix one, log a new entry that **corrects** it; the report marks the old one as corrected.

Side effects

- The mould card shows **Last work** (type, names, date).

- Logging work on a mould with an open problem **resolves the problem** (audit `problem.resolved`).
- Audit `worklog.created` / `worklog.corrected`.

> Screenshot needed: Work log form with type Repair, two people and one photo.

Evidence

`backend/lib/workLog.js` `planWorkLog`; `backend/controllers/workLogController.js` `createWorkLog`; `backend/models/WorkLog.js`; `backend/lib/images.js` `photosFromDataUrls`; `backend/test/workLog.test.js`; `frontend/src/components/WorkLogForm.tsx`.

Workshop queue and status changes

See what needs workshop attention and change a mould's condition at the Workshop

Audience: Mould Maker, Mould Technician, Admins · Status: Verified (static code review + tests) · Source: docs/user-guide/workshop-queue.md

Availability

- Page `/workshop` (**Workshop queue**) requires `moulds.view`, so everyone can see it.
- **Change status** requires `workshop.work`: Super Admin, Admin, Mould Technician, Mould Maker.

What the queue shows

The page makes three requests and shows:

Section	Source
Problem reports (count badge)	moulds with an open problem (`problem=true`)
At the Workshop	`location = WORKSHOP`, with "since …"
Flagged elsewhere (send to Workshop)	Needs/In service or repair but not at the Workshop

If all are empty: "Nothing waiting. 👍"

Change status — steps

1. Open a mould **at the Workshop**.
2. Select **Change status**, choose the **New status** and enter **Remarks** (required).
3. Select **Confirm**.

Rules

- Only at the Workshop (`NOT_AT_WORKSHOP`); the status must differ (`NO_CHANGE`).
- **In service → Good** resets `totalShots` to 0 and sets **Last serviced**.
- Any other status → **Good** while the shot limit is reached is refused (`SERVICE_DUE`): service it first.
- Writes a `STATUS_CHANGED` history line and an audit entry.

Statuses

Status	Meaning	Allowed locations
Good	Ready to use	Anywhere
Needs service / Needs repair	Flagged	Machine, Production Floor, Workshop, Out of Service (never storage)
In service / In repair	Being worked on	Workshop or Out of Service only

> Screenshot needed: `/workshop` with moulds in both sections.

Evidence

`frontend/src/app/(tracker)/workshop/page.tsx`; `backend/lib/workflow.js` `planStatus`; `backend/lib/mouldState.js` `mouldStateError`.

Roles

Roles overview

The six roles, their responsibilities and daily journeys

Audience: Everyone; managers · Status: Verified (roles from code); responsibilities and escalation paths require product-owner confirmation · Source: docs/roles/overview.md

The code defines exactly six roles (`backend/lib/permissions.js` `ROLES`). Legacy roles `operator` and `viewer` are mapped to Mould Technician and Production Supervisor until the upgrade migration rewrites them.

Role	Code	In one line	Page
Super Admin	`super_admin`	Everything except custom access; owns racks, audit and Out of Service	super-admin.md
Admin	`admin`	Manages moulds, machines, users and reports; racks read-only	admin.md
Production Supervisor	`production_supervisor`	Moves moulds on machines/floor, records shots, reports problems	production-supervisor.md
Mould Technician	`mould_technician` (default for new users)	Machine and workshop work, logs work	mould-technician.md
Mould Maker	`mould_maker`	Storage moves, workshop work, logs work	mould-maker.md
System Administrator	`system_admin`	All permissions; protected, invisible account	system-administrator.md

Full detail: permissions-matrix.md.

Typical handoffs (derived from permissions)

Diagram (Mermaid source; shown as a picture in the website version)

```
sequenceDiagram
    participant MM as Mould Maker
    participant MT as Mould Technician
    participant PS as Production Supervisor
    participant A as Admin
    MM->>MM: Rack -> Production Floor
    MT->>MT: Floor -> Machine (mount)
    PS->>PS: Record shots / report problem
    MT->>MT: Machine -> Workshop (Needs repair)
    MM->>MM: In repair -> Good (work logged)
    MM->>MM: Workshop -> rack
    A->>A: Work log report / export
```

Separation of duties (as implemented)

- Production Supervisors cannot log work or change workshop status; they report problems.
- Only Super Admins change the rack layout, send moulds Out of Service and read the audit log.
- Only System Administrators manage custom access, and their accounts are hidden from everyone else.
- Admins cannot touch Super Admin accounts.

Requires product-owner confirmation: the shift-level escalation path (who to call when a move is refused).

Permissions matrix

Every permission by role, and who may make each move

Audience: Admins, System Administrators, auditors · Status: Verified (backend/lib/permissions.js + backend/test/routes.test.js, workflow.test.js) · Source: docs/roles/permissions-matrix.md

Source of truth: `backend/lib/permissions.js` (`PERMISSIONS`, `can`). The frontend mirrors it only to show or hide buttons (`frontend/src/utils/permissions.ts`). Every API route is checked against every role by `backend/test/routes.test.js`.

Legend: ☑ allowed by role default · — not allowed. **SysA** (System Administrator) has every permission. Custom access (user-access) can replace a non-SysA user's defaults.

By permission

Permission	What it allows	Super Admin	Admin	Prod. Supervisor	Mould Technician	Mould Maker	SysA
`moulds.view`	View moulds, dies, machines, workshop queue	☑	☑	☑	☑	☑	☑
`racks.view`	Rack view	☑	☑	☑	☑	☑	☑
`worklog.view`	View a mould's work log	☑	☑	☑	☑	☑	☑
`problem.report`	Report a problem	☑	☑	☑	☑	☑	☑
`moves.machine`	Mount / move on machines, to floor; record shots	☑	☑	☑	☑	—	☑
`moves.mouldRoom`	Move into/out of Mould Room / Tool Room	☑	☑	—	—	☑	☑
`workshop.work`	Send to Workshop, change status, record shots	☑	☑	—	☑	☑	☑
`worklog.create`	Log work	☑	☑	—	☑	☑	☑
`moulds.manage`	Add/edit/delete moulds, items, machines	☑	☑	—	—	—	☑
`machines.override`	Mount on a non-listed machine (with reason)	☑	☑	—	—	—	☑
`users.manage`	Manage users	☑	☑ (not Super Admins)	—	—	—	☑
`worklog.report`	Work log report, summary, export	☑	☑	—	—	—	☑
`racks.manage`	Change zones and racks	☑	— (view only)	—	—	—	☑
`audit.view`	Audit log	☑	—	—	—	—	☑
`moves.outOfService`	Send out / return from Out of Service	☑	—	—	—	—	☑
`access.manage`	Custom user access	—	—	—	—	—	☑
`settings.manage`	*Defined, no route uses it*	☑	—	—	—	—	☑
`import.run`	*Defined, no route uses it (imports are CLI scripts)*	☑	—	—	—	—	☑
`export.run`	*Defined, no route uses it (export uses `worklog.report`)*	☑	—	—	—	—	☑

Move matrix

A move needs one permission from **every** group that applies (`movePermissionGroups`). The result by role:

From → To	Super Admin	Admin	Prod. Supervisor	Mould Technician	Mould Maker
Storage → Machine	☑	☑	—	—	—
Storage → Production Floor	☑	☑	—	—	☑
Storage → Workshop	☑	☑	—	—	☑
Storage → other rack	☑	☑	—	—	☑
Machine/Floor → Machine	☑	☑	☑	☑	—
Machine → Floor	☑	☑	☑	☑	—
Machine/Floor → Storage	☑	☑	—	—	☑
Machine/Floor → Workshop	☑	☑	—	☑	☑
Workshop → Machine	☑	☑	☑	☑	—
Workshop → Floor	☑	☑	☑	☑	☑
Workshop → Storage	☑	☑	—	—	☑
Any ↔ Out of Service	☑	—	—	—	—

"Storage" means the Mould Room (moulds) or the Tool Room (dies). Location rules (status Good, free machine, …) apply on top; see Move a mould.

Enforcement

- Backend: `requirePermission` on every route (`backend/routes/table.js`), plus `canMove` / `can` inside the workflow functions.
- Each request reloads the user from the database, so role and active changes apply at once (`backend/middleware/authMiddleware.js`).
- Unauthorized: HTTP 403 "You do not have permission to do this" / `FORBIDDEN_MOVE`.

Super Admin (`super_admin`)

Responsibilities and daily activities of the Super Admin role

Audience: Super Admins · Status: Verified (permissions); daily routine is a suggestion · Source: docs/roles/super-admin.md

Can do

Everything Admin can, plus: change zones and racks, read the audit log, send moulds Out of Service and return them, and manage Admins and other Super Admins. Cannot manage custom access or see System Administrator accounts.

Typical journey

Sign in → **Workshop queue** (anything stuck?) → **Audit log** (yesterday's overrides and deletes) → **Rack management** (new racks) → sign out.

Pages

All sidebar pages except **User access**.

Related: Permissions matrix · Out of Service · Rack management

Admin (`admin`)

Responsibilities and daily activities of the Admin role

Audience: Admins · Status: Verified (permissions); daily routine is a suggestion · Source: docs/roles/admin.md

Can do

Every move except Out of Service; mount on a non-listed machine with a reason; add/edit/delete/restore moulds and items; edit machines; manage users (not Super Admins); log work; work log report and export. Rack management is **view only**. No audit log.

Typical journey

Sign in → **Find a mould** → add or correct moulds/items → **Machines** → **Users** (new starters) → **Work log report** (weekly export) → sign out.

Related: Manage moulds · Users · Work log report

Production Supervisor (`production_supervisor`)

Responsibilities and daily activities of the Production Supervisor role

Audience: Production Supervisors · Status: Verified (permissions); daily routine is a suggestion · Source: docs/roles/production-supervisor.md

Can do

View everything operational; mount a mould from the floor or the Workshop onto a machine, change machines, move it machine → floor; record shots; report problems.

Cannot do

Move into or out of storage, send to the Workshop, change status, log work, or manage anything.

Typical journey

Sign in → **Find a mould** (search by machine code) → **Record shots** at shift end → **Report a problem** if something is wrong → hand off to the Mould Technician → sign out.

Related: Report a problem · Record shots

Mould Technician (`mould_technician`) — default role for new users

Responsibilities and daily activities of the Mould Technician role

Audience: Mould Technicians · Status: Verified (permissions); daily routine is a suggestion · Source: docs/roles/mould-technician.md

Can do

Machine moves (not from or to storage), send to the Workshop, change status at the Workshop, record shots, log work (with up to 4 co-workers), report problems.

Cannot do

Move into or out of the Mould Room / Tool Room, override machine capability, or manage anything.

Typical journey

Sign in → **Workshop queue** (flagged elsewhere) → mount / unmount on machines → **Send to Workshop** with remarks → log the work with outcome → sign out.

Related: Move a mould · Work log

Mould Maker (`mould_maker`)

Responsibilities and daily activities of the Mould Maker role

Audience: Mould Makers · Status: Verified (permissions); daily routine is a suggestion · Source: docs/roles/mould-maker.md

Can do

Moves into and out of the Mould Room / Tool Room (rack changes, storage → floor/workshop, back to a rack), send to the Workshop, change status, Workshop → floor, log work, report problems.

Cannot do

Mount on machines or record shots (needs `moves.machine`), or manage anything.

Typical journey

Sign in → **Workshop queue** → set **In repair** / **In service** → log work → mark **Good** → put it back on a rack (**Rack view** to find space) → sign out.

Related: Workshop queue · Workshop repair cycle

System Administrator (`system_admin`)

The protected all-permission role for IT

Audience: IT / System Administrators · Status: Verified (static code review + tests) · Source: docs/roles/system-administrator.md

Properties

- Holds every permission, including `access.manage` (User access).
- **Protected:** the account is visible only to its owner, even to other System Administrators. Super Admins and Admins cannot list, edit, delete or assign this role.
- In history, reports and exports seen by others, the person appears as **Private user**. Audit entries involving the account are excluded from other users' results.

Provisioning

No default credentials exist. Set `SYSTEM_ADMIN_NAME`, `SYSTEM_ADMIN_EMAIL` and `SYSTEM_ADMIN_PASSWORD` in the backend's local environment, then run `npm run create:system-admin` in `backend/`. The command only creates a new account; it never promotes or overwrites an existing one.

Evidence

`backend/lib/permissions.js` (`canViewUser`, `canManageUser`); `backend/lib/accountPrivacy.js`; `backend/middleware/accountPrivacy.js`; `backend/scripts/create-system-admin.mjs`; root `README.md`.

Workflows

Workflows overview

Inventory of verified end-to-end workflows and the location/status state machine

Audience: All users; managers; testers · Status: Verified (backend/lib/workflow.js, backend/lib/workLog.js + tests) · Source: docs/workflows/overview.md

Workflow	Trigger	Actors	Page
Mould registration and retirement	New mould / mould no longer used	Admin	mould-lifecycle.md
Mount and unmount	Production needs a mould	Maker, Technician, Supervisor, Admin	mount-and-unmount.md
Shot count and service	Shift production	Supervisor, Technician	shot-count-and-service.md
Problem → repair	Someone sees a defect	Anyone → Technician/Maker	problem-to-repair.md
Workshop repair cycle	Mould needs service/repair	Technician, Maker	workshop-repair-cycle.md
Out of Service (external repair)	Repair outside the factory	Super Admin	out-of-service.md
Data import and migration	Initial load / workbook update	IT (CLI)	data-import.md

Not found in the code: approvals, scheduled maintenance plans, notifications, archival beyond soft delete.

The two state rules (always enforced)

1. A mould that is **not Good** may only be at the Workshop, on a machine, on the Production Floor or Out of Service. It is **never in storage**.
2. **In service / In repair** may only be at the Workshop or Out of Service.

These rules are checked by the database model on every save (`mouldStateError`), not just by the UI.

Diagram (Mermaid source; shown as a picture in the website version)

```
stateDiagram-v2
    direction LR
    state "Mould Room / Tool Room<br/>(Good only)" as S
    state "Machine<br/>(one per machine)" as M
    state "Production Floor" as F
    state "Workshop" as W
    state "Out of Service" as O
    S --> M: Admin only (two permissions)
    S --> F
    S --> W
    M --> F
    M --> M: change machine
    F --> M
    M --> S: Good only
    F --> S: Good only
    M --> W
    F --> W
    W --> M: Good only
    W --> F: Good only
    W --> S: Good only
    W --> O: Super Admin
    O --> W: Super Admin
```

Concurrency and failure handling

- **Optimistic locking:** writes carry the mould's `version`. If it changed, the result is `409 STALE` ("…was changed by someone else. Refresh and try again.").
- **Transactions:** each change and its History + Audit lines are saved together or not at all (`withTransaction`, `backend/config/db.js`).
- **Database guards:** a unique index allows one mould per machine; Tool IDs are unique.
- **Double submits:** the UI disables buttons while saving (`rack-admin`, `users` pages).
- **Offline:** no offline queue; actions need the server.

Mould registration and retirement

Add a mould, maintain it, soft-delete and restore it

Audience: Admin, Super Admin · Status: Verified · Source: docs/workflows/mould-lifecycle.md

Trigger: a new mould arrives, or one is retired. **Actor:** Admin / Super Admin (`moulds.manage`).

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart TD
    A[Add mould: Tool ID, type, items] --> B{Starts at}
    B -->|Mould Room| C[Choose rack; owner zone must match or give reason]
    B -->|Tool Room / Floor / Workshop| D[Saved]
    C --> D[Saved as Good + CREATED history + audit]
    D --> E[In use...]
    E --> F{Retire?}
    F -->|On a machine| G[Refused: take it off first]
    F -->|Elsewhere| H[Delete with reason: soft delete]
    H --> I[Hidden from lists; history kept]
    I -->|Restore| D
```

Outputs: History `CREATED` / `DELETED` / `RESTORED`; audit `mould.created` / `mould.deleted` / `mould.restored`. **Failure paths:** `DUPLICATE_TOOL_ID`, `RACK_REQUIRED`, `RACK_FULL`, `OWNER_MISMATCH`, `UNMOUNT_FIRST`.

Details: Manage moulds.

Mount and unmount

Get a mould from storage onto a machine and back again

Audience: Mould Maker, Mould Technician, Production Supervisor, Admin · Status: Verified (backend/test/workflow.test.js) · Source: docs/workflows/mount-and-unmount.md

Trigger: production needs a mould on a machine.

Who does which step

Only Admins and Super Admins hold both `moves.mouldRoom` and `moves.machine`, so only they can go rack → machine directly. On the shop floor the move is a **two-person handoff**:

Diagram (Mermaid source; shown as a picture in the website version)

```
sequenceDiagram
    actor MM as Mould Maker
    actor MT as Technician / Supervisor
    participant API
    MM->>API: move to PRODUCTION_FLOOR (from rack)
    API-->>MM: OK, history MOVED
    MT->>API: move to MACHINE (machineId)
    alt machine occupied
        API-->>MT: 409 MACHINE_OCCUPIED
    else not listed for machine
        API-->>MT: 409 NOT_CAPABLE (Admin may override with reason)
    else shot limit reached
        API-->>MT: 409 SERVICE_DUE
    else
        API-->>MT: OK
    end
    Note over MT: ...production, Record shots...
    MT->>API: move to PRODUCTION_FLOOR
    MM->>API: move to MOULD_ROOM (rackId)
```

Decision points

- Status must be **Good** (`NOT_GOOD`).
- The machine must be active and free; the mould must be listed for it or overridden.
- When going back to a rack: the rack must be active and not full; if the owner zone differs, a reason is required.

Audit implications

Every move: `Movement` (from/to location, machine code, rack location ID, override reason) + `AuditLog` `mould.moved`.

Steps per screen: Move a mould.

Shot count and service

How shots accumulate, when a mould becomes service-due, and how the count resets

Audience: Supervisors, Technicians, Admins · Status: Verified (backend/lib/workflow.js rule 8) · Source: docs/workflows/shot-count-and-service.md

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart TD
    A[Mounted, Good] -->|Record shots| B{totalShots >= shotsToService?}
    B -->|No| A
    B -->|Yes| C[Auto: Needs service + history line]
    C --> D[Send to Workshop]
    D --> E[Change status: In service]
    E --> F[Change status: Good]
    F --> G[totalShots = 0, Last serviced = now]
    H[Work log: type Service, outcome OK] --> G
```

Rules

- Only applies when **Shots to service** is set (> 0).
- Auto-flag happens only for **Good** moulds **outside storage**. In storage the status stays Good, but mounting is refused with `SERVICE_DUE`.
- Setting **Good** directly from Needs/In repair while still over the limit is refused; it must go through **In service**.
- A work-log entry of type **Service** with outcome **OK** also resets the count, wherever the mould is.
- The stats endpoint counts `serviceDueNotFlagged` (over the limit but still Good, for example in storage).

Evidence: `planShots`, `serviceDueChange`, `planStatus` in `backend/lib/workflow.js`; `planWorkLog` in `backend/lib/workLog.js`; `mouldStats` in `backend/controllers/mouldController.js`.

Problem to repair

From a problem report to a resolved, repaired mould

Audience: All roles · Status: Verified · Source: docs/workflows/problem-to-repair.md

Diagram (Mermaid source; shown as a picture in the website version)

```
sequenceDiagram
    actor PS as Anyone (e.g. Supervisor)
    actor MT as Technician / Maker
    participant M as Mould
    PS->>M: Report a problem (text, photos)
    Note over M: openProblem set, status/location unchanged, shown on Workshop queue
    alt Fix in place
        MT->>M: Log work (e.g. Minor fix, outcome OK)
    else Needs workshop
        MT->>M: Send to Workshop (remarks) or log work with Needs repair + Send to Workshop
    end
    Note over M: openProblem cleared, audit problem.resolved
```

Failure paths: a Supervisor cannot move to the Workshop; they hand over to a Technician/Maker. When a Needs… outcome is logged while the mould is in storage, the entry is saved with warning `WORKSHOP_MOVE_REQUIRED`.

Related: Report a problem · Work log

Workshop repair cycle

Send to the Workshop, repair or service, mark Good, return to use

Audience: Mould Technician, Mould Maker, Admins · Status: Verified · Source: docs/workflows/workshop-repair-cycle.md

Step	Actor	Action	Result
1	Technician / Maker	**Send to Workshop**, optional New status, **Remarks** required	Location Workshop; latest remarks shown; open problem resolved
2	Technician / Maker	**Change status** → In repair / In service (remarks)	`STATUS_CHANGED`
3	Technician / Maker	Log work (type, parts, time, people, photos)	Work log entry; "Last work" on the card
4	Technician / Maker	**Change status** → Good (remarks)	From In service: shots reset
5	Maker (to rack) or Technician/Supervisor (to machine)	Move out	Only Good moulds leave

Diagram (Mermaid source; shown as a picture in the website version)

```
stateDiagram-v2
    [*] --> NeedsRepair: flagged / sent
    NeedsRepair --> InRepair
    NeedsService --> InService
    InRepair --> Good
    InService --> Good: resets shots
    NeedsRepair --> Good: if not over shot limit
    Good --> [*]: leaves Workshop
```

Related: Workshop queue

Out of Service

Send a mould outside the factory and return it to the Workshop

Audience: Super Admin · Status: Verified (static code review) · Source: docs/workflows/out-of-service.md

Actor: Super Admin only (`moves.outOfService` cannot be granted to anyone else).

Send out

1. Open the mould → **Send out of service**.
2. Enter **Recipient / Company name** (max 150), **Address** (max 500) and **Repair details** (max 2000; also used as remarks).
3. Confirm. The details are stored with `sentAt`, both on the mould and in the movement history.

Return

1. Open the mould → **Return to Workshop**. Remarks are required.
2. `returnedAt` is stamped. The mould arrives at the **Workshop**; from there the normal repair cycle applies.

Rules

- The status cannot be changed while Out of Service (`OUT_OF_SERVICE_STATUS`): return it first.
- The dispatch details are a snapshot. Later dispatches never rewrite earlier history.

Evidence: `backend/lib/workflow.js` `planMove`; `backend/models/DispatchDetails.js`; `backend/lib/outOfService.js`; `frontend/src/components/DispatchDetails.tsx`.

Data import and migration

Command-line scripts that load and upgrade data

Audience: IT, developers · Status: Verified (script headers + tests); not run in this documentation pass · Source: docs/workflows/data-import.md

There is **no import screen**; imports are CLI scripts run from `backend/`. All are **dry runs unless `--apply`** is given.

Script	Purpose	Key behaviour
`node scripts/import-moulds.mjs --file <xlsx> [--sheet …] [--apply] [--match "SHEET=DB"]`	Import moulds, items and pictures from the tools-list workbook	Tool ID match ignoring spaces → sheet values win, history stays. Near matches held back unless `--match`. **Nothing is deleted.**
`node scripts/attach-cleaned-pictures.mjs --dir <folder> [--apply]`	Point items at background-removed pictures	Originals kept for rollback
`node migrations/upgradeMouldModel.mjs [--apply]`	One-time copy from the old shared DB (`SOURCE_MONGODB_URI`, read-only) into this app's DB, upgrading Tool → Mould/Item/Movement and roles	Run once; recorded in `MigrationRun`
`node migrations/moveExtrusionToToolRoom.mjs [--apply]`	Move the 20 imported extrusion dies to the Tool Room	Only eligible dies
`migrations/importLegacy.mjs`	Legacy import (used by the upgrade)	Tested in `legacyMigration.test.js`
`npm run seed`	Development seed data	**Do not run against production**
`npm run migrate:supabase`	Old Supabase migration	Historic

> Production safety: the live database is shared with another app. Never delete existing data; take a backup before any `--apply`. See Operations.

Evidence: script header comments; `backend/test/importMoulds.test.js`, `attachCleaned.test.js`, `legacyMigration.test.js`.

Demonstrations

5-minute tour

Scripted short demo for managers and new users

Audience: Presenter; audience = managers, new staff · Status: Script derived from verified features; not rehearsed on a running instance · Source: docs/demonstrations/five-minute-tour.md

Prerequisites: demo data, signed in as **admin@demo.local**, on a phone or a narrow window.

#	Screen	Narrator script	Clicks	Expected result	Value
1	`/find`	"Every mould, where it is and its condition, live."	Type `DEMO`	3 cards, status badges, locations	One source of truth
2	Detail sheet	"Here's what this mould makes and its full history."	Tap `DEMO100`	Items, shots, history	Traceability
3	Move	"Moving is guarded by rules."	**Mount on a machine** → `DEMO-02` (not listed)	Admin override reason requested	Prevents wrong machine
4	Move	—	Choose `DEMO-01` → Confirm	Card shows the machine; history line added	Audit trail
5	`/workshop`	"What needs attention right now."	Open Workshop queue	`DEMO200` at the Workshop	Maintenance visibility
6	`/racks`	"The Mould Room at a glance."	Rack view, zone DMA	Empty / in use / full colours	Find space fast

Cleanup: none needed (demo DB), or move `DEMO100` back to `DMA-A1`.

15-minute operational demo

Full shop-floor cycle across roles

Audience: Presenter; audience = supervisors, technicians, makers · Status: Script derived from verified features; not rehearsed · Source: docs/demonstrations/fifteen-minute-demo.md

Use two browsers (or one normal and one private window) to switch roles quickly.

#	Role	Action	Expected result
1	Maker	`DEMO100` → **To Production Floor**	Location Production Floor
2	Supervisor	Try **Put in Mould Room**	Not offered (no storage permission)
3	Technician	**Mount on a machine** → `DEMO-01`	On DEMO-01
4	Supervisor	**Record shots** 1,200	Auto **Needs service**; "Service due"; history "Shot limit reached (1200/1000 shots)"
5	Technician	Try **Mount** on DEMO-02	Refused: only to the Workshop
6	Supervisor	**Report a problem** "Flash on cavity 2" + photo	Problem badge; on the Workshop queue; status unchanged
7	Technician	**Send to Workshop**, remarks "Service + flash"	At the Workshop; problem resolved
8	Maker	**Change status** → In service (remarks)	In service
9	Maker	Log work: Service, 45 min, parts "O-ring", outcome OK, done by Maker + Technician	Good, shots reset to 0, Last serviced today
10	Maker	**Put in Mould Room** → `DMB-A1`	Owner mismatch → give a reason → stored
11	Admin	**Work log report**, range Today → export XLSX	File with "Work log" + "About this export" sheets

Business value: enforced rules, no lost moulds, service by usage, full accountability. Reset: demo-data-and-reset.md.

30-minute administrator / technical demo

Administration, security and architecture walkthrough

Audience: Presenter; audience = IT, admins, auditors · Status: Script derived from verified features; not rehearsed · Source: docs/demonstrations/admin-technical-demo.md

Min	Topic	Show	Point out
0–5	Architecture	architecture.md diagram, `compose.yaml`	Mongo on an internal network; health checks
5–10	Rack management (Super Admin)	Add zone `DMC`, bulk generate A–A 1–3 with Preview, rename code	Preview counts; `rack.renamed` audit
10–14	Users	Create a Technician; as Admin try to edit a Super Admin	Refused
14–18	User access (System Administrator)	Give the Technician custom access without `workshop.work`, then Reset	Takes effect on the next request
18–22	Audit log	Filter Racks, Users	Before/after, reason, append-only
22–25	Concurrency	Edit the same mould in two windows	Second save → `STALE`
25–28	API	`GET /api/health`; `POST /api/moulds/:id/move` returning `MACHINE_OCCUPIED` JSON	Error envelope, codes
28–30	Tests	`cd backend && npm test`	157 tests including every route × role

Optional interactive tour (proposal, not implemented)

Step	Anchor (to verify)	Title	Role	Body
T1	search input on `/find`	Find any mould	all	Search by Tool ID, item, machine or rack
T2	first filter chip	Filter	all	Tap a chip to filter
T3	first card	Open	all	Tap to see details and actions
T4	Workshop link	Workshop queue	workshop roles	What needs attention

Requirements: Next/Back/Skip, completion stored per user, keyboard accessible, respects reduced motion.

Demo data and reset

Safe, fictional dataset and environment for demonstrations

Audience: Presenters, IT · Status: Planned / proposed (dataset spec); seed script exists but its contents were not reviewed for demo fitness · Source: docs/demonstrations/demo-data-and-reset.md

> **Never demo on production.** The production database is shared with another app and holds real data.

Environment

A disposable local stack (local development) or a separate Compose project with its own volumes and its own `MONGODB_URI`.

Accounts (create via Users; fictional)

Email	Role
super@demo.local	Super Admin
admin@demo.local	Admin
supervisor@demo.local	Production Supervisor
tech@demo.local	Mould Technician
maker@demo.local	Mould Maker

Use demo-only passwords; do not reuse real ones.

Dataset (proposed)

Kind	Values
Zones	`DMA` (Demo A), `DMB` (Demo B)
Racks	Bulk generate `DMA` rows A–B, numbers 1–4, capacity 2
Machines	`DEMO-01`, `DEMO-02` (create via API `POST /api/machines`, or the seed)
Moulds	`DEMO100` (rack DMA-A1, owner DMA, capable DEMO-01, shots to service 1000), `DEMO200` (Workshop, Needs repair), `DEMO300` (DEMO-02), die `DIE900` (Tool Room)
Items	`DEMO100` → item `ITM-100` "Demo cup", 4 cavities

Reset

Drop the **demo** database only (or `docker compose -p demo down -v`), then recreate. Because history is append-only, resetting means a fresh database, not undoing actions.

Administration

Configuration

Environment variables for backend, frontend and the Compose stack

Audience: IT, developers · Status: Verified (.env examples, index.js, compose.yaml); values intentionally omitted · Source: docs/administration/configuration.md

No real values are shown. Keep secrets out of Git (in production they are in Coolify's environment settings).

Backend (`backend/.env`, see `backend/.env.example`)

Variable	Required	Description
`MONGODB_URI`	Yes	This app's database (replica set)
`SOURCE_MONGODB_URI`	Migration only	Old database, read-only, for `upgradeMouldModel.mjs`
`PORT`	No (4000)	API port
`JWT_SECRET`	Yes, ≥32 chars	Signs session tokens; startup fails otherwise
`JWT_EXPIRES_IN`	No (`7d`)	Token lifetime. The cookie `maxAge` is fixed at 7 days.
`CORS_ORIGIN`	Yes	Comma-separated allowed frontend origins
`NODE_ENV`	No	`production` hides internal error messages
`COOKIE_SECURE`	No (`false`)	`true` when served over HTTPS
`COOKIE_SAME_SITE`	No (`lax`)	Cookie SameSite
`WEBAUTHN_ORIGIN`	No	Exact HTTPS origin; blank disables passkeys
`WEBAUTHN_RP_ID`	No	Hostname of `WEBAUTHN_ORIGIN`
`UPLOAD_DIR`	No (`backend/uploads`)	Picture storage
`SYSTEM_ADMIN_NAME` / `_EMAIL` / `_PASSWORD`	For `create:system-admin` only	Provision the first System Administrator

Root `.env` for Compose (`.env.production.example`)

`MONGO_ROOT_PASSWORD`, `MONGO_APP_PASSWORD`, `JWT_SECRET` (generate each with `openssl rand -hex 32`), `APP_ORIGIN` (exact browser URL; becomes `CORS_ORIGIN`), `HTTP_PORT` (default 8081), `COOKIE_SECURE`, `WEBAUTHN_ORIGIN`, `WEBAUTHN_RP_ID`, optional `NPM_REGISTRY`.

Frontend

`frontend/.env.local.example`: the API base used by the browser in development.

In-app configuration

Setting	Where	Who
Zones and racks	Rack management	Super Admin
Machines (name, active)	Machines	Admin
Users, roles	Users	Admin / Super Admin
Custom permissions	User access	System Administrator

Setting	Where	Who
Language, theme	Per user / device	Everyone

There is no settings screen; `settings.manage` is defined but unused.

Operations and recovery

Health checks, logs, backup, restore and incident notes

Audience: IT · Status: Health/logs verified from code; backup procedure partially verified (deploy/README.md) · Source: docs/administration/operations-and-recovery.md

Health checks

- `GET /api/health` → `database: "connected"`. Compose uses this as the backend health check (30 s interval).
- MongoDB health: `deploy/mongo-health.js`.
- The UI sync indicator shows Live / Offline per user.

Logs

`docker compose logs --tail=100` (standalone), or the Coolify service logs. The API logs unexpected errors to stdout. There is no log shipping or monitoring integration.

Backup (must cover both)

1. **MongoDB:** a `mongodump --archive --gzip` of the app database. A pre-Coolify backup example exists (see `deploy/README.md`).
2. **Uploads folder** (`/app/uploads` in the backend container): pictures. See risk R4 in Security.

Take a backup before every deploy that includes a migration or an import `--apply`.

> The live database is shared with another application. **Never drop or delete existing collections or data**; restores must target only this app's database.

Restore (outline — confirm with IT)

1. Stop the backend.
2. `mongorestore --archive --gzip` into this app's database only.
3. Restore the uploads folder.
4. Start the backend; check `/api/health`; sign in and open a mould with pictures.

Incident quick reference

Symptom	Check	Action
Everyone "Offline"	Proxy/backend container up? `/api/health`	Restart the service in Coolify
Health says `disconnected`	Mongo container / replica set	Check mongo logs; `rs.status()`
Pictures broken after redeploy	Uploads folder empty	Restore uploads; add a volume
Passkeys stopped working	Hostname/origin changed	`WEBAUTHN_ORIGIN`/`RP_ID` must match; re-enrol
One user can't do something	Role / custom access	Users or User access page

Disaster recovery

No documented RPO/RTO. *Requires owner confirmation* (see GAPS).

Security and access

Existing security controls, privacy, auditability, risks and recommendations

Audience: IT, auditors, System Administrators · Status: Verified controls (static review); recommendations clearly separated · Source: docs/administration/security-and-access.md

Existing controls (verified)

Control	Implementation	Evidence
Password hashing	bcrypt, cost 12	`authController.js`
Session	JWT in an HttpOnly cookie; SameSite lax; Secure when `COOKIE_SECURE=true`	`utils/session.js`
Per-request account check	User reloaded each request; inactive or deleted → 401	`authMiddleware.js`
Authorization	Permission per route plus rule-level checks; all routes × all roles tested	`routes/table.js`, `test/routes.test.js`
Input validation	Zod on every body and query; JSON size limits 1 MB / 8 MB	`validators/schemas.js`, `index.js`
Rate limiting	Login/passkey 40 per 10 min, enrolment 20 per 10 min	`authRoutes.js`
HTTP headers	Helmet; CORS allow-list with credentials	`index.js`
Passkeys	WebAuthn; public key only; origin check; current password required to add or remove	`passkeyController.js`, `passkeyConfig.js`
Uploads	Only JPEG/PNG/WebP decoded by Sharp, re-encoded to WebP; served to signed-in users only; dotfiles denied	`lib/images.js`, `index.js`
Audit trail	Append-only AuditLog, Movement, WorkLog	`lib/immutable.js`
Protected accounts	System Administrators hidden; shown as "Private user"	`lib/accountPrivacy.js`
Network	MongoDB on an internal Docker network only; keyfile auth	`compose.yaml`
Errors	Internal messages hidden in production	`errorMiddleware.js`

Known risks / observations

#	Observation	Impact
R1	Logout clears the cookie but does not revoke the JWT; a copied token stays valid until expiry unless the user is disabled	Low–medium
R2	Cookie `maxAge` is hard-coded to 7 days regardless of `JWT_EXPIRES_IN`	Config confusion
R3	No account lockout beyond IP rate limiting	Low
R4	`/api/uploads` has no volume in `compose.yaml`	Data loss risk (pictures)
R5	The Windows installer is unsigned	Users must trust the source
R6	`deploy/README.md` "Passkeys and role permissions" still describes old roles (admin/operator/viewer)	Wrong guidance

Recommended improvements (not implemented)

- Token revocation or a session version on the user, so password change or logout invalidates sessions.

- Tie cookie `maxAge` to `JWT_EXPIRES_IN`.
- Add a persistent volume for uploads and include it in backups.
- Update the outdated role text in `deploy/README.md` and the root `README.md`.

Data privacy

Personal data stored: user name, email, role; names in work-log snapshots. There is no retention or purge process. Customer data: Out of Service recipient name and address.

Technical

Architecture

Components, boundaries and request flow

Audience: Developers, architects · Status: Verified · Source: docs/technical/architecture.md

Diagram (Mermaid source; shown as a picture in the website version)

```
flowchart LR
    subgraph Client
      B[Next.js pages - client components]
      SW[Service worker]
    end
    subgraph Server [Docker Compose]
      NG[Nginx proxy :80]
      FE[frontend: next start]
      API[backend: Express]
      DB[(mongo:7 replica set rs0)]
      UP[(/app/uploads folder)]
    end
    B --> NG
    NG --> FE
    NG -->|/api| API
    API --> DB
    API --> UP
```

Backend layering

Layer	Folder	Responsibility
Routes	`routes/`	Route tables `[method, path, permissions, handler, bodySchema, querySchema]`, built by `routerFrom` (auth → permission → validation → privacy → handler)
Validation	`validators/schemas.js`	Zod schemas
Controllers	`controllers/`	Load documents, call pure rules, save in a transaction with Movement + AuditLog
Rules (pure)	`lib/workflow.js`, `lib/workLog.js`, `lib/mouldState.js`, `lib/permissions.js`	All business decisions; unit-tested without a DB
Models	`models/`	Mongoose schemas, invariants, indexes, immutability plugin
Privacy	`middleware/accountPrivacy.js`, `lib/accountPrivacy.js`	Replaces protected users with "Private user" in every JSON response

Frontend

Next.js App Router. Route group `(tracker)` shares `AppLayout` (session check, sidebar, 60 s permission refresh). Data via `utils/api.ts` (fetch with credentials). Permissions mirrored in `utils/permissions.ts` for display only. i18n in `utils/i18n.ts` (en, si). The PWA service worker is `public/sw.js`.

Authentication flow

Diagram (Mermaid source; shown as a picture in the website version)

```
sequenceDiagram
    Browser->>API: POST /api/auth/login {email,password}
    API->>DB: find user, bcrypt compare
    API-->>Browser: Set-Cookie access_token (JWT sub=userId, HttpOnly, SameSite=lax)
```

```
    Browser->>API: any /api call (cookie)
    API->>DB: load user (active, not deleted) each request
    API->>API: can(user, permission)
```

Key design decisions (from code comments)

- Rules are pure functions so they can be tested exhaustively (`backend/test/workflow.test.js`).
- History stores text snapshots (machine code, rack ID, person name) so renames never rewrite the past.
- Pictures are content-hashed WebP files, cached immutably.

Repository structure

Map of folders and key files

Audience: Developers · Status: Verified · Source: docs/technical/repository-structure.md

```
tool-tracker/
├── backend/                 Express API (package tool-tracker-backend 2.0.0)
│   ├── index.js             server entry, middleware, route mounting, /api/health
│   ├── config/db.js         Mongo connection + withTransaction
│   ├── routes/              route tables (table.js builds routers)
│   ├── controllers/         auth, passkey, mould, workLog, rack, machine, user, audit
│   ├── lib/                 pure rules: workflow, workLog, mouldState, permissions, racks,
images, audit…
│   ├── middleware/          auth, validate, errors, accountPrivacy
│   ├── models/              Mongoose schemas
│   ├── validators/          Zod schemas
│   ├── migrations/          one-time data migrations
│   ├── scripts/             import-moulds, attach-cleaned-pictures, create-system-admin
│   ├── test/                node:test suites (157 tests)
│   └── Dockerfile
├── frontend/                Next.js 16 app (tool-tracker-frontend 2.0.0)
│   ├── src/app/             login, (tracker)/<page>/page.tsx, manifest
│   ├── src/components/      AppLayout, Sidebar, MouldSearch, MouldSheet, MouldActions, forms…
│   ├── src/utils/           api, i18n, permissions, helpers, imageUpload
│   ├── public/              sw.js, offline.html, install.html, icons
│   ├── test/                node:test suites (29 tests)
│   └── Dockerfile
├── native/                  desktop (Electron), android (WebView), apple (source only)
├── deploy/                  nginx configs, mongo init/health, deployment and HTTPS guides
├── compose.yaml             production stack
├── .env.production.example  root env template (no values)
├── archive/, output/, outputs/  previous versions, build outputs
└── docs/                    this documentation
```

The root `README.md` still describes the pre-upgrade Tool model in places; see GAPS.

Local development

Run, test and build the app on a developer machine

Audience: Developers · Status: Verified (tests run 2026-10-29); dev servers not started in this pass · Source: docs/technical/local-development.md

Prerequisites

Node.js (a version supporting `node --test` and Next 16), a MongoDB **replica set** (Atlas or local `--replSet`), Git.

Backend

```
cd backend
cp .env.example .env      # fill MONGODB_URI, JWT_SECRET (≥32 chars), CORS_ORIGIN
npm install
npm run seed              # development data only - never against production
npm run dev               # nodemon, http://localhost:4000/api/health
npm test                  # node --test  -> 157 tests
```

The server refuses to start without a `JWT_SECRET` of at least 32 characters.

Frontend

```
cd frontend
cp .env.local.example .env.local
npm install
npm run dev               # http://localhost:3000
npm run typecheck
npm test                  # 29 tests (confirm, imageUpload, service worker, theme)
npm run build
```

Passkeys locally

`WEBAUTHN_ORIGIN=http://localhost:3000`, `WEBAUTHN_RP_ID=localhost` (development only).

Test results at documentation time

Suite	Result
backend `npm test`	157 pass, 0 fail (19 suites)
frontend `npm test`	29 pass, 0 fail

Contribution rules (observed conventions)

- Business rules go in pure `lib/` functions with tests; controllers only load and save.
- New routes go in a route table with explicit permissions; `routes.test.js` checks every role.
- Every change writes Movement and/or AuditLog in the same transaction.
- Update `docs/` and `DOCUMENTATION-CHANGELOG.md` with any behaviour change.

Database reference

MongoDB collections, fields, enums, indexes and invariants

Audience: Developers, DBAs · Status: Verified against backend/models/.js · Source: docs/technical/database-reference.md*

MongoDB via Mongoose 8. Transactions are required (replica set). Schema files: `backend/models/`.

Diagram (Mermaid source; shown as a picture in the website version)

```
erDiagram
    MOULD ||--o{ ITEM : makes
    MOULD }o--o| MACHINE : "mounted on"
    MOULD }o--o| RACK : "stored on"
    ZONE ||--o{ RACK : contains
    MOULD ||--o{ MOVEMENT : history
    MOULD ||--o{ WORKLOG : work
    USER ||--o{ MOVEMENT : performedBy
    USER ||--o{ PASSKEY : owns
    USER ||--o{ AUDITLOG : actor
```

Mould

Field	Type	Notes
toolId	String	required, unique, uppercase, ≤100
toolType	enum	`INJECTION_MOULD` (default), `EXTRUSION_DIE`
toolSetCount, size	String	≤50
owner	String	zone code, uppercase, ≤10
capableMachines	[String]	machine codes
status	enum	`GOOD, NEEDS_SERVICE, NEEDS_REPAIR, IN_SERVICE, IN_REPAIR`
location	enum	`MACHINE, PRODUCTION_FLOOR, WORKSHOP, MOULD_ROOM, TOOL_ROOM, OUT_OF_SERVICE`; validated by `mouldStateError`
machineId / rackId	ObjectId	only with MACHINE / MOULD_ROOM
awaitingRack	Boolean	imported, no rack yet
shotsToService, totalShots, lastServicedAt	Number/Date	rule 8
notes, remarks	String ≤2000	remarks = latest service/repair note
outOfServiceDetails	DispatchDetails	recipientName, address, repairDetails, sentAt, returnedAt
lastMovedAt/By, lastWork, openProblem		denormalised for cards
deletedAt/By		soft delete
version	Number	optimistic lock

Indexes: unique `machineId` where `location = MACHINE` (one mould per machine); `capableMachines`; `openProblem.at`; status, location, rackId, deletedAt.

Item

`mouldId, itemId` (unique per mould), description, cavities, qas, materialId/Description, masterbatchId/Description, `images[{path, thumbPath, width, height, source}]`, notes, sortOrder, deletedAt.

Machine

`code` (unique, uppercase), `name`, `active`.

Zone / Rack

- Zone: `code` (unique, `[A-Z0-9]{1,10}`), name, active, sortOrder, deletedAt.
- Rack: zone, row (`[A-Z]{1,3}`), number (1–999), code, `locationId` (unique, `ZONE-CODE`), capacity (null = unlimited), active, note, deletedAt.

Movement (append-only)

mouldId, toolId, type (`CREATED, EDITED, MOVED, STATUS_CHANGED, DELETED, RESTORED, IMPORTED`), from/to location, status, machineId+code snapshot, rackId+locationId snapshot, outOfServiceDetails, remarks, overrideReason, performedBy, createdAt.

WorkLog (append-only)

mouldId, toolId, type (`REPAIR, SERVICE, MINOR_FIX, INSPECTION, CLEANING, OTHER`), description, partsUsed, timeSpentMinutes, doneBy[1–5]{userId,name,role}, recordedBy, doneAt, place snapshot, statusBefore/After, shots, outcome (`OK, NEEDS_REPAIR, NEEDS_SERVICE`), sentToWorkshop, photos[≤3], correctsId, problemReported, resolvesProblemId. Indexes on doneAt plus mould, person, type, machine and problem.

AuditLog (append-only)

actorId (null = system), actorName, action, entityType, entityId, entityLabel, before, after, reason, ip, createdAt.

User

name, email (unique, lowercase), passwordHash (bcrypt, cost 12), role, permissionOverrides (null = role defaults), language (`en`/`si`), active, deletedAt.

Passkey / AuthChallenge

Passkey: userId, credentialId (unique), publicKey, counter, transports, name, deviceType, backedUp, lastUsedAt. AuthChallenge: short-lived WebAuthn challenges.

Legacy / support

`Tool`, `ToolMovement` (pre-upgrade model, kept for migration), `MigrationRun` (one-time migration record).

Retention

Nothing is hard-deleted by the app: moulds, items, racks, zones and users are soft-deleted; history, work log and audit are append-only (`backend/lib/immutable.js`). No retention or purge job exists.

API reference

Every REST endpoint with permissions, inputs, side effects and errors

Audience: Developers, integrators, testers · Status: Verified against backend/routes/.js and backend/validators/schemas.js · Source: docs/technical/api-reference.md*

Base path `/api`. JSON only. Authentication uses the HttpOnly cookie `access_token` set by login. All routes built with `routerFrom` require sign-in, then **any one** of the listed permissions (`backend/routes/table.js`).

Response envelope

```
{ "success": true, "...": "..." }
{ "success": false, "message": "Human readable", "code": "MACHINE_OCCUPIED", "details": {
"occupant": "FLK19066" } }
```

HTTP	When
400	Validation (Zod), `NO_CHANGE`, `REMARKS_REQUIRED`, bad id, invalid JSON
401	No / invalid / expired session; user inactive or deleted
403	Missing permission, `FORBIDDEN_MOVE`
404	Not found / route not found
409	Business-rule conflict (`STALE`, `NOT_GOOD`, `MACHINE_OCCUPIED`, `DUPLICATE`, …)
410	`/api/tools` (old API, replaced)
413	`TOO_LARGE` body (1 MB; 8 MB on work log / problem routes)
429	Rate limit on auth / passkey routes
500	Unexpected; message hidden in production

Health

Method	Path	Auth	Returns
GET	`/health`	none	`{ service, database: "connected"|"disconnected", timestamp }`

Auth — `/api/auth` (`backend/routes/authRoutes.js`)

Method	Path	Auth	Body	Notes
POST	`/login`	rate-limited 40/10 min	`{ email, password }`	Sets cookie (7 days)
POST	`/logout`	—	—	Clears cookie
GET	`/me`	signed in	—	User + `permissions[]`
PATCH	`/me`	signed in, rate-limited	`{ name, email, currentPassword, newPassword? }`	Profile
PATCH	`/me/language`	signed in	`{ language: "en"|"si" }`	
POST	`/password`	signed in	`{ currentPassword, newPassword }`	
GET	`/passkeys/capabilities`	—	—	Whether passkeys are configured
GET	`/passkeys`	signed in	—	Your passkeys

Method	Path	Auth	Body	Notes
POST	`/passkeys/register/options`	signed in, 20/10 min, origin check	`{ currentPassword, name }`	WebAuthn options
POST	`/passkeys/register/verify`	signed in, origin check	`{ response }`	
POST	`/passkeys/login/options`	40/10 min, origin check	—	
POST	`/passkeys/login/verify`	40/10 min, origin check	`{ response }`	Sets cookie
DELETE	`/passkeys/:id`	signed in, origin check	`{ currentPassword }`	

Moulds — `/api/moulds` (`backend/routes/mouldRoutes.js`)

Method	Path	Permission (any)	Body / query	Side effects
GET	`/`	moulds.view	`toolType, q, location, status, owner, zone, rackId, machineId, awaitingRack, needsService, problem, deleted`	—
GET	`/stats`	moulds.view	`toolType`	—
GET	`/:id`	moulds.view	—	Deleted only for moulds.manage
GET	`/:id/history`	moulds.view	—	—
POST	`/`	moulds.manage	`toolId, toolType?, toolSetCount?, size?, owner?, capableMachines?, shotsToService?, notes?, location, rackId?, overrideReason, items[]`	CREATED + audit
PATCH	`/:id`	moulds.manage	mould fields + `remarks, version`	EDITED + audit
DELETE	`/:id`	moulds.manage	`{ reason }`	Soft delete
POST	`/:id/restore`	moulds.manage	—	RESTORED
POST	`/:id/move`	moves.outOfService, moves.machine, moves.mouldRoom, workshop.work (exact check in `planMove`)	`to, machineId?/machineCode?, rackId?, status?, remarks, overrideReason, outOfServiceDetails?, version`	MOVED + audit; may resolve problem
POST	`/:id/status`	workshop.work	`status, remarks, version`	STATUS_CHANGED
POST	`/:id/shots`	moves.machine, workshop.work	`shots (1–10,000,000), version`	May auto-flag NEEDS_SERVICE
POST	`/:id/items`	moulds.manage	item fields	item.created
PATCH	`/:id/items/:itemId`	moulds.manage	item fields + `photo` (data URL)	item.updated
DELETE	`/:id/items/:itemId`	moulds.manage	`{ reason }`	item.deleted
GET	`/:id/worklogs`	worklog.view	—	—
POST	`/:id/worklogs`	worklog.create	see work log	WorkLog + audit; may move / change status
POST	`/:id/problems`	problem.report	`description, doneAt?, photos[≤3], version`	openProblem

Example move:

```
POST /api/moulds/6650c0ffee0000000000abcd/move
Content-Type: application/json

{ "to": "MACHINE", "machineCode": "INJ-03", "remarks": "", "version": 7 }
```

Racks — `/api/racks`

Method	Path	Permission	Body
GET	`/`	racks.view	— (zones, racks, occupancy)
GET	`/:id`	racks.view	—
POST	`/zones`	racks.manage	`{ code, name, sortOrder }`
PATCH	`/zones/:id`	racks.manage	`{ code?, name?, active?, sortOrder? }`
DELETE	`/zones/:id`	racks.manage	—
POST	`/`	racks.manage	`{ zone, row, number, capacity, note }`
PATCH	`/:id`	racks.manage	`{ row?, number?, capacity?, active?, note? }`
DELETE	`/:id`	racks.manage	—
POST	`/bulk`	racks.manage	`{ zone, rowFrom, rowTo, from, to, capacity, preview=true }`
POST	`/remove-row`	racks.manage	`{ zone, row }`

Machines — `/api/machines`

Method	Path	Permission	Body
GET	`/`	moulds.view	—
POST	`/`	moulds.manage	`{ code, name }`
PATCH	`/:id`	moulds.manage	`{ name?, active? }`

Users — `/api/users`

Method	Path	Permission	Body
GET	`/`	users.manage	—
POST	`/`	users.manage	`{ name, email, password, role }`
PATCH	`/:id`	users.manage	`{ email?, name?, role?, active?, password? }`
DELETE	`/:id`	users.manage	`{ reason }`
GET	`/access`	access.manage	—
PATCH	`/:id/access`	access.manage	`{ permissions: string[] | null }`

Work logs — `/api/worklogs`

Method	Path	Permission	Query
GET	`/workers`	worklog.create	— (people who may be named)
GET	`/`	worklog.report	`q, toolId, userId, type, machine, outcome, status, problem, range, from, to, page, limit≤100`

Method	Path	Permission	Query
GET	`/summary`	worklog.report	same filters
GET	`/export`	worklog.report	same + `format=csv|xlsx` (cap 5,000)

Audit — `/api/audit`

Method	Path	Permission	Query
GET	`/`	audit.view	`entityType, entityId, action, actorId, from, to, page, limit≤200`

Uploads

`GET /api/uploads/<path>`: signed-in users only; immutable cache for 365 days (file names are content hashes).

Error codes (business)

`NOT_FOUND, STALE, NO_CHANGE, MOULD_DELETED, INVALID_LOCATION, FORBIDDEN_MOVE, FORBIDDEN, STATUS_ONLY_AT_WORKSHOP, INVALID_STATUS, OUT_OF_SERVICE_STATUS, REMARKS_REQUIRED, DISPATCH_DETAILS_REQUIRED, INVALID_DISPATCH_DETAILS, NOT_GOOD, MACHINE_REQUIRED, MACHINE_INACTIVE, SERVICE_DUE, MACHINE_OCCUPIED, NOT_CAPABLE, OVERRIDE_REASON_REQUIRED, RACK_REQUIRED, RACK_INACTIVE, RACK_FULL, OWNER_MISMATCH, NOT_AT_WORKSHOP, NOT_ON_MACHINE, INVALID_SHOTS, DUPLICATE, DUPLICATE_TOOL_ID, UNMOUNT_FIRST, RACK_OCCUPIED, INVALID_RANGE, INVALID_PHOTO, TOO_LARGE, DESCRIPTION_REQUIRED, DONE_AT_FUTURE, INVALID_DATE, TOO_MANY_WORKERS, INVALID_WORKER, INVALID_CORRECTION, EXPORT_TOO_LARGE, GONE`. Warnings (successful response): `CAPABILITY_OVERRIDDEN, OWNER_ZONE_OVERRIDDEN, SERVICE_STILL_DUE, WORKSHOP_MOVE_NOT_ALLOWED, WORKSHOP_MOVE_REQUIRED`.

Total: **55 routed endpoints** (1 health + 13 auth + 17 moulds + 10 racks + 3 machines + 6 users + 4 work logs + 1 audit), plus the `/api/uploads` static files and the `/api/tools` 410 stub.

Integrations

External systems the app talks to

Audience: Developers, IT · Status: Verified (absence confirmed by dependency and code review) · Source: docs/technical/integrations.md

Integration	Present?	Notes
MongoDB	Yes	Only data store
WebAuthn (browser platform authenticators)	Yes	`@simplewebauthn/server` / `browser`
File export (CSV / XLSX)	Yes	Download only, work log report
Excel workbook import	Yes, CLI	`scripts/import-moulds.mjs` (ExcelJS)
Email / SMS / push notifications	Not found	
ERP / MES / external API	Not found	
Webhooks, message queues, scheduled jobs	Not found	
QR / barcode	Not found	Old hash links (`#/page/m/<id>`) are redirected and may have been used in QR codes; unconfirmed
Old Supabase store	Historic	`migrateSupabase.js`

Deployment

Production stack, environment, release and rollback

Audience: IT, developers · Status: Verified from compose.yaml and deploy/README.md; not executed in this pass · Source: docs/technical/deployment.md

The authoritative step-by-step guide is `deploy/README.md`, with HTTPS in `deploy/LOCAL-HTTPS.md`. This page summarises them.

Stack (`compose.yaml`)

Service	Image	Role
mongo	mongo:7.0	Replica set `rs0`, keyfile auth, internal network only
mongo-init	mongo:7.0	Creates the app user (one-shot)
backend	./backend	API on 4000, health check `/api/health` must report `connected`
frontend	./frontend	Next.js
proxy	nginx:stable-alpine	Port `${HTTP_PORT:-8081}` → 80

Volumes `mongo-data`, `mongo-config`. **Never delete these volumes.**

> ⚠ **Risk:** `compose.yaml` declares no volume for the backend's `/app/uploads` (item and work-log pictures, created in `backend/Dockerfile`). Unless the Coolify service adds one, pictures are lost when the backend container is recreated. Confirm this; see GAPS Q1.

Current production

Managed by **Coolify** on the factory server. Build the backend/frontend images from the source folder, then redeploy in Coolify. Secrets live in the Coolify environment settings. Do not start a second standalone copy. (The host, paths and URL are in `deploy/README.md`; they are not repeated here.)

Release procedure

1. Run `npm test` in `backend` and `frontend`; run `npm run build` in `frontend`.
2. **Back up the database** (see Operations).
3. Run any required migration as a dry run, then with `--apply`.
4. Build the images and redeploy.
5. Check `/api/health` and sign in.

Rollback

Redeploy the previous images. The data model is append-only, so code rollback does not need data rollback unless a migration ran; then restore the backup taken in step 2.

Native shells

- Windows: `native/desktop` → `npm ci && npm run dist:win` (unsigned NSIS installer).
- Android: `python native/android/build.py` (Java 21, Android SDK). The signing key is kept outside the repo; **back it up**.
- Bump the version for shell updates. Web UI updates need no reinstall.

Support

Troubleshooting

Symptom, cause, check, fix and escalation for common problems

Audience: All users; support · Status: Verified (messages taken from source) · Source: docs/support/troubleshooting.md

Message / symptom	Cause	Fix	Escalate to
"Invalid email or password"	Wrong credentials or disabled account	Retry; check caps lock	Admin
"Too many authentication attempts…"	40 attempts in 10 min	Wait 10 minutes	—
"Invalid or expired session" / sent to sign-in	Session expired, user disabled	Sign in again	Admin
"You do not have permission to do this"	Role or custom access	Ask someone with the right role	Admin / System Administrator
"…is needs repair; it can only go to the Workshop" (`NOT_GOOD`)	Only Good moulds leave the Workshop	Send to Workshop, fix, mark Good	Technician / Maker
"…already has X; take it off first" (`MACHINE_OCCUPIED`)	One mould per machine	Move X off first	Supervisor
"…is not listed for INJ-03" (`NOT_CAPABLE`)	Machine not in capable machines	Choose a listed machine, or an Admin overrides with a reason	Admin
"…has reached its shot limit…" (`SERVICE_DUE`)	Shots ≥ limit	Service at the Workshop	Technician
"Rack FLK-A3 is full" (`RACK_FULL`)	Capacity reached	Choose another rack	Super Admin (capacity)
"…belongs to FLK, not CCI; give a reason…" (`OWNER_MISMATCH`)	Owner zone ≠ rack zone	Use own zone, or give a reason	—
"Remarks are required…"	Workshop / Out of Service moves and status changes	Enter remarks	—
"Send … to the Workshop before changing its status"	Status changes only at the Workshop	Move first	—
"…was changed by someone else. Refresh and try again." (`STALE`)	Concurrent edit	Reload and redo	—
"Take … off the machine before deleting it"	Delete on a machine	Unmount first	—
"…entries match; narrow the filters to 5000 or fewer"	Export cap	Narrow the date range	—
"Upload is too large" / "Photos must be JPEG, PNG or WebP"	Photo size/type	Use a smaller JPEG/PNG/WebP	—
"The work time cannot be in the future"	Device clock / date field	Fix the date	—
"Offline — cannot reach server"	Network / server down	Check Wi-Fi/LAN; retry	IT
Passkey button disabled	No trusted HTTPS configuration	Use password	IT
Certificate warning on a device	Factory CA not trusted	Install the CA (`deploy/LOCAL-HTTPS.md`)	IT

FAQ

Frequently asked questions

Audience: All users · Status: Verified · Source: docs/support/faq.md

Why can't I put a mould from a rack straight onto a machine? That needs both storage and machine permissions, which only Admins and Super Admins have. On the floor, a Mould Maker brings it to the Production Floor and a Technician or Supervisor mounts it. See Mount and unmount.

Can I edit or delete a work-log entry? No. Log a correction entry instead; the old one is marked corrected.

Does reporting a problem stop the machine or change the status? No. It only flags the mould and lists it on the Workshop queue.

When is a problem closed? When someone logs work on the mould or sends it to the Workshop.

Is a deleted mould gone? No. It is hidden but keeps its history; an Admin can restore it.

Does the app work offline? It shows an offline page, but changes need the server.

Who can see the audit log? Super Admins and System Administrators.

Why does a name show as "Private user"? It belongs to a protected System Administrator account.

How often does the list refresh? Every 10 seconds.

Can I use Sinhala? Yes: the EN / සිං toggle. The choice follows your account.

Can I scan a QR code? No scanner is built in. Old `#/…/m/<id>` links still open the right mould.

Known limitations

Current behaviour limits, separated from recommendations and roadmap

Audience: Everyone; product owner · Status: Verified · Source: docs/support/known-limitations.md

Current behaviour

- No offline editing or queued changes.
- No notifications (email, push, SMS) for service-due moulds or problems.
- No QR/barcode scanning or label printing.
- No scheduled/preventive maintenance by date; service is driven by shot count only.
- Shots are entered manually (no machine counter integration).
- The mould list is not paginated; all matches load (fine at the current size, may slow with thousands).
- No UI to add a machine (API only).
- No import screen; imports are CLI scripts.
- `settings.manage`, `import.run` and `export.run` permissions exist but do nothing.
- Passkeys don't work in the native shells.
- Windows installer unsigned; no iOS/macOS release.
- Work-log date ranges use the Asia/Colombo time zone.
- Work-log export capped at 5,000 rows.

Recommended improvements

See security-and-access.md and GAPS.

Future roadmap

Not defined in the repository. *Requires product-owner input.*

Evidence

Evidence matrix

Traceability of documented claims to source code and tests

Audience: Auditors, maintainers · Status: Verified · Source: docs/analysis/EVIDENCE-MATRIX.md

Verification methods: **S** = static code review · **T** = covered by an automated test that passed on 2026-10-29 · **D** = from repository documentation · **R** = runtime (not performed in this pass).

#	Claim	Status	Source	Route / endpoint	Method	Notes
1	6 roles exist	Verified	`backend/lib/permissions.js` `ROLES`	—	S,T	Legacy operator/viewer aliased
2	19 permissions and role mapping	Verified	`permissions.js` `PERMISSIONS`	all	S,T	`routes.test.js`
3	System Administrator has all permissions and is hidden	Verified	`permissions.js` `can`, `canViewUser`; `lib/accountPrivacy.js`	`/api/users`	S,T	`auth.test.js`
4	`access.manage` cannot be delegated	Verified	`permissions.js` `can`	`/api/users/access`	S	
5	Out of Service only for Super Admin	Verified	`permissions.js` `can`; `userController.updateUserAccess`	move	S	
6	Each route requires sign-in + a permission	Verified	`routes/table.js`	all	S,T	
7	User reloaded on every request	Verified	`middleware/authMiddleware.js`	all	S	
8	Session = JWT in an HttpOnly cookie, 7 days	Verified	`utils/session.js`	`/api/auth/login`	S	
9	Login rate limit 40/10 min	Verified	`routes/authRoutes.js`	`/api/auth/login`	S	
10	Passkeys (WebAuthn) with password confirmation	Verified	`controllers/passkeyController.js`, `validators/schemas.js`	`/api/auth/passkeys/*`	S,T	
11	Locations and statuses (enums)	Verified	`lib/mouldState.js`	—	S,T	
12	Non-Good never in storage; In service/repair only at the Workshop/OOS	Verified	`mouldState.js` `mouldStateError`	—	S,T	`mouldModel.test.js`
13	Move permission groups	Verified	`lib/workflow.js` `movePermissionGroups`	`POST /moulds/:id/move`	S,T	`workflow.test.js` "who may move what"
14	One mould per machine (DB index)	Verified	`models/Mould.js` partial unique index	move	S,T	
15	Capable machine override for Admins with reason	Verified	`workflow.js` `planMove`	move	S,T	

#	Claim	Status	Source	Route / endpoint	Method	Notes
16	Rack capacity / inactive / owner-zone reason	Verified	`planMove`	move	S,T	
17	Remarks required to Workshop / OOS / status change	Verified	`planMove`, `planStatus`	move, status	S,T	
18	Shot limit auto-flags Needs service	Verified	`planShots`, `serviceDueChange`	`POST /:id/shots`	S,T	
19	In service → Good resets shots	Verified	`planStatus`	status	S,T	
20	Extrusion dies stored in the Tool Room	Verified	`mouldState.js` `storageLocation`	—	S,T	`toolRoom.test.js`
21	Optimistic locking `STALE`	Verified	`mouldController.js` `loadForUpdate`	PATCH/move/status	S	
22	Changes + history + audit in one transaction	Verified	`config/db.js` `withTransaction`; controllers	—	S	
23	Soft delete / restore moulds; can't delete when mounted	Verified	`deleteMould`, `restoreMould`	DELETE/restore	S	
24	Search fields and spacing-insensitive Tool ID	Verified	`listMoulds`	`GET /moulds`	S	
25	Stats counts	Verified	`mouldStats`	`GET /moulds/stats`	S	
26	Work-log fields, 1–5 people, ≤3 photos	Verified	`models/WorkLog.js`, `schemas.js`	`POST /:id/worklogs`	S,T	
27	Outcome effects and warnings	Verified	`lib/workLog.js` `planWorkLog`	worklogs	S,T	`workLog.test.js`
28	Work log append-only; corrections via `correctsId`	Verified	`lib/immutable.js`, `WorkLog.js`	—	S,T	
29	Problem report never changes status/location	Verified	`planProblemReport`	`POST /:id/problems`	S,T	
30	Problem resolved by work log or Workshop move	Verified	`workLogController.resolveProblem`, `moveMould`	—	S	
31	Report filters, Asia/Colombo days, paging	Verified	`schemas.js` `workLogQuery`, `lib/dateRange.js`	`GET /worklogs`	S,T	`workLogReport.test.js`
32	Export CSV/XLSX, cap 5,000, audited	Verified	`exportWorkLogs`, `lib/workLogFilter.js`	`GET /worklogs/export`	S,T	
33	Zones/racks CRUD, bulk preview, occupied guards	Verified	`rackController.js`	`/api/racks/*`	S	
34	Machines: no disable while occupied; code fixed in UI	Verified	`machineController.js`; `machines/page.tsx`	`/api/machines`	S	

#	Claim	Status	Source	Route / endpoint	Method	Notes
35	No add-machine UI	Verified	`machines/page.tsx` (only list/update calls)	—	S	
36	User management rules	Verified	`userController.js`, `canManageUser`	`/api/users`	S,T	
37	Custom access replaces role defaults	Verified	`can` (`permissionOverrides`)	`/api/users/:id/access`	S	
38	Audit log append-only, filters, ≤200/page	Verified	`AuditLog.js`, `schemas.js` `auditQuery`	`GET /audit`	S	
39	Uploads: JPEG/PNG/WebP → WebP 800 px + 200 px thumbnail	Verified	`lib/images.js`	uploads	S,T	`itemPhoto.test.js`
40	Uploads served to signed-in users only	Verified	`index.js`	`/api/uploads`	S	
41	10 s list refresh; 60 s permission refresh	Verified	`MouldSearch.tsx`, `AppLayout.tsx`	—	S	
42	English + Sinhala UI	Verified	`utils/i18n.ts`	—	S	
43	PWA + offline page	Verified	`public/sw.js`, `manifest.ts`	—	S,T	`service-worker.test.mjs`
44	Legacy hash links redirect	Verified	`utils/helpers.ts` `getLegacyDestination`	`/`	S	
45	Native shells (Windows, Android), Apple source only	Verified	`native/README.md`	—	D,S	
46	Compose stack, Mongo replica set	Verified	`compose.yaml`	—	S	
47	Production on Coolify	Partially verified	`deploy/README.md`	—	D	Server not inspected
48	No uploads volume in Compose	Verified	`compose.yaml`, `backend/Dockerfile`	—	S	Coolify config not inspected
49	`settings.manage`, `import.run`, `export.run` unused	Verified	grep of `backend/`	—	S	
50	No notifications / QR / scheduler / integrations	Not found	dependencies, code search	—	S	
51	Import scripts dry-run by default, nothing deleted	Verified	script headers	CLI	S,T	
52	UI walkthrough steps and button labels	Partially verified	`i18n.ts`, components	pages	S	Not exercised at runtime (R)

Coverage summary

	Count
Verified	49
Partially verified	2 (#47, #52)
Not found (confirmed absent)	1 (#50)

Gaps and open questions

Uncertainties, inconsistencies and questions for the product owner

Audience: Product owner, IT, maintainers · Status: Open · Source: docs/analysis/GAPS-AND-QUESTIONS.md

Questions for the owner

#	Question	Why it matters
Q1	Does the Coolify service mount a persistent volume on the backend's `/app/uploads`? `compose.yaml` has none.	Item and work-log pictures could be lost on redeploy and are not in Mongo backups
Q2	What is the official version number? Packages say 2.0.0; native shells say 2.2.0.	Version labels in the docs
Q3	Should `settings.manage`, `import.run` and `export.run` be removed, or are screens planned?	They appear in User access but have no effect
Q4	Is an "Add machine" button wanted on `/machines`? (The API supports it.)	New machines currently need API/scripts
Q5	Escalation paths per shift (who to call when a move is refused)?	Roles pages
Q6	Backup schedule, RPO/RTO and the restore owner?	Operations page
Q7	Is a Sinhala quick-start document wanted? (The UI already supports Sinhala.)	Optional deliverable
Q8	May screenshots be captured from production (with redaction), or only from a demo stack?	Screenshot backlog
Q9	Are QR labels in use with the old `#/…/m/<id>` links?	Integrations / FAQ

Inconsistencies found in existing docs (not changed)

File	Issue
`README.md` (root)	Top half describes the pre-upgrade Tool app (roles admin/operator/viewer, `/api/tools`, a seed with default credentials, "auto refresh every 10s"). The `/api/tools` route now returns 410.
`deploy/README.md` § "Passkeys and role permissions"	Describes Admin/Operator/Viewer; the current roles are different
`backend/.env.example`	`JWT_EXPIRES_IN` exists, but the cookie `maxAge` is hard-coded to 7 days

Not verified at runtime

- The UI flows were not clicked through; the app was not started against a database.
- The Mermaid diagrams were not rendered by a tool (syntax checked by review).
- Native builds were not built.

Screenshot backlog

Every `Screenshot needed:` line in `docs/` (at least 12). Capture them from the demo stack and save under `docs/assets/screenshots/`.

Documentation changelog

Revision log for the docs folder

Audience: Maintainers · Status: Current · Source: docs/DOCUMENTATION-CHANGELOG.md

Date	Version	Change	Basis
2026-10-29	1.0	Initial documentation suite created (orientation, getting started, 17 feature pages, 8 roles pages, 8 workflows, 4 demos, administration, technical reference, support, evidence matrix, gaps, complete guide). Existing READMEs left unchanged; their inconsistencies are listed in GAPS.	Branch `feature/mould-tracker-upgrade`, static review; backend 157/157 and frontend 29/29 tests passing

Maintenance rules

- Update the affected page **and** the evidence matrix whenever `backend/lib/permissions.js`, `lib/workflow.js`, `lib/workLog.js`, routes or schemas change.
- Bump `last_verified` only after re-checking against the code.
- Keep proposals labelled "Planned / proposed".